

고급머신러닝 보고서

옵티마이저

Optimizer

전자공학부 전자공학과
2020144005 김윤성
2024. 10. 08 제출

목차 INDEX

01 옵티마이저란

- 옵티마이저
- SGD의 장점과 단점
- 옵티마이저의 종류

02 옵티마이저의 종류

- 주요 옵티마이저들
- 안장점
- 기타 옵티마이저들

03 실습 #1)

- DNN by keras
- 코드 분석
- 실행 결과

04 실습 #2)

- Optimizer 비교
- 코드 분석
- 실행 결과
- 분석 내용

01

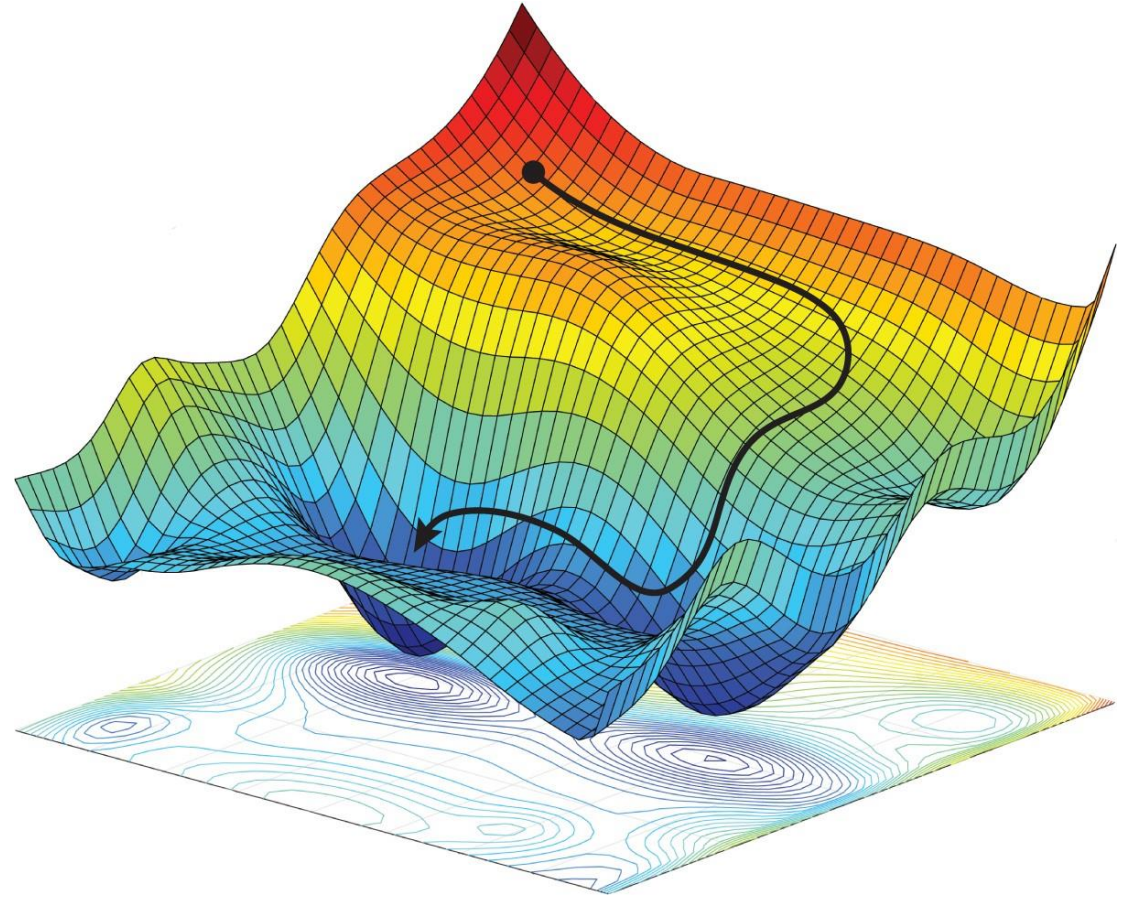
옵티마이저란

Optimizer

옵티마이저

Optimizer

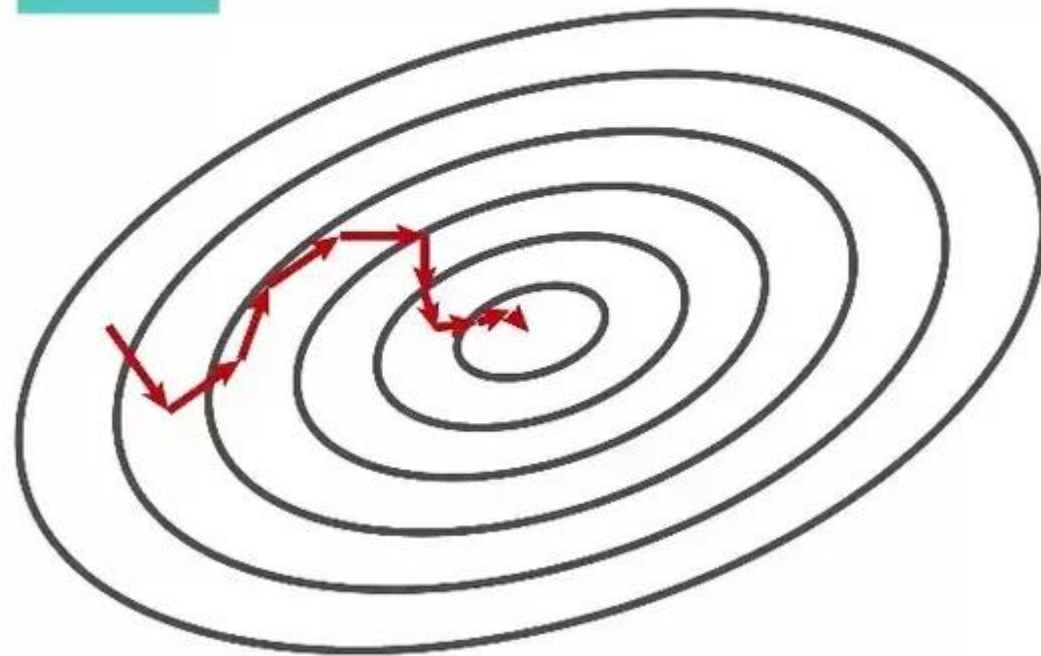
- Cost function을 최소화시키는 Parameter를 찾는 알고리즘
- Cost function을 그래프로 그렸을 때, 가장 아래에 있는 점으로 이동하게 하는 알고리즘



SGD Stochastic Gradient Descent

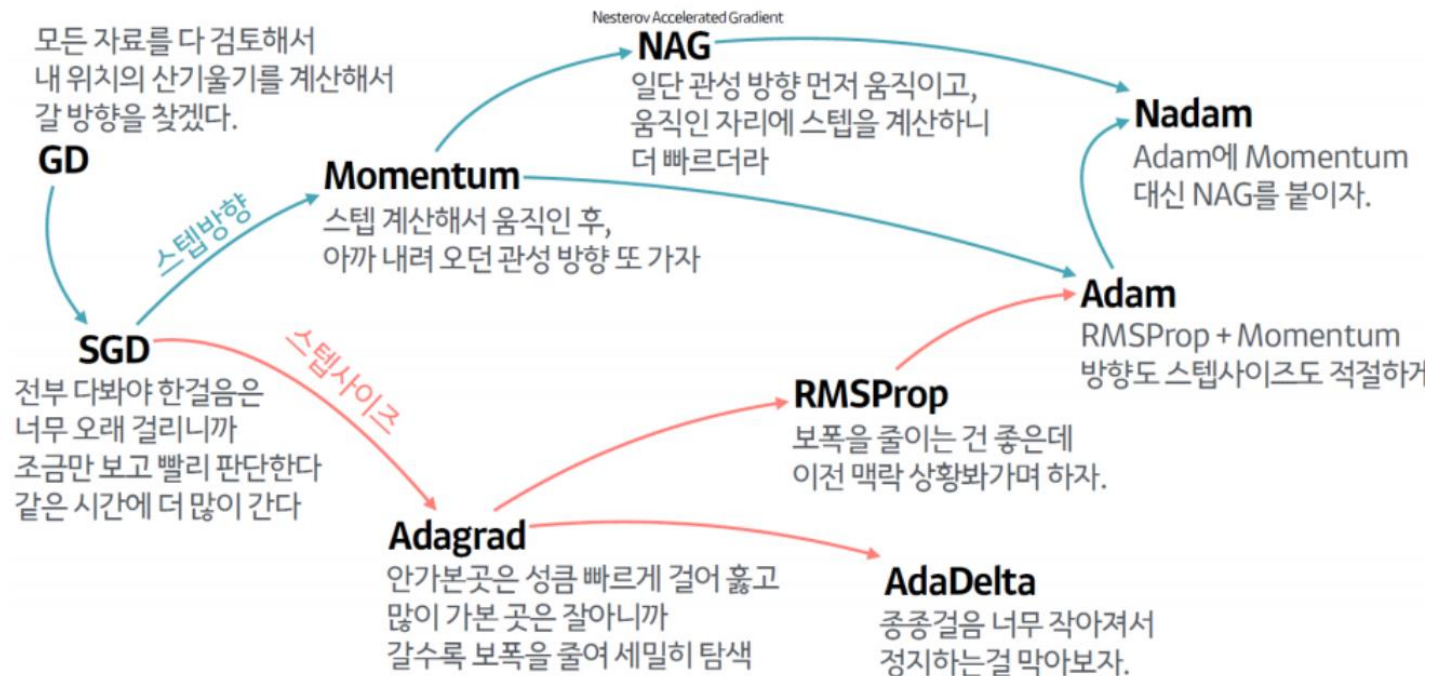
- 일반 GD에 비해 더 빠르게 수렴 가능
- 구조가 비교적 간단해 입문하기 쉬움
- 요즘 옵티마이저에 비하면 수렴 속도가 느림
- Local Minima에 빠질 위험이 높음

Stochastic Gradient Descent



옵티마이저 Optimizer

- 앞의 문제를 해결하기 위해 수많은 옵티마이저가 개발됨
- 요즘은 Adam계열 옵티마이저가 주로 사용됨



02

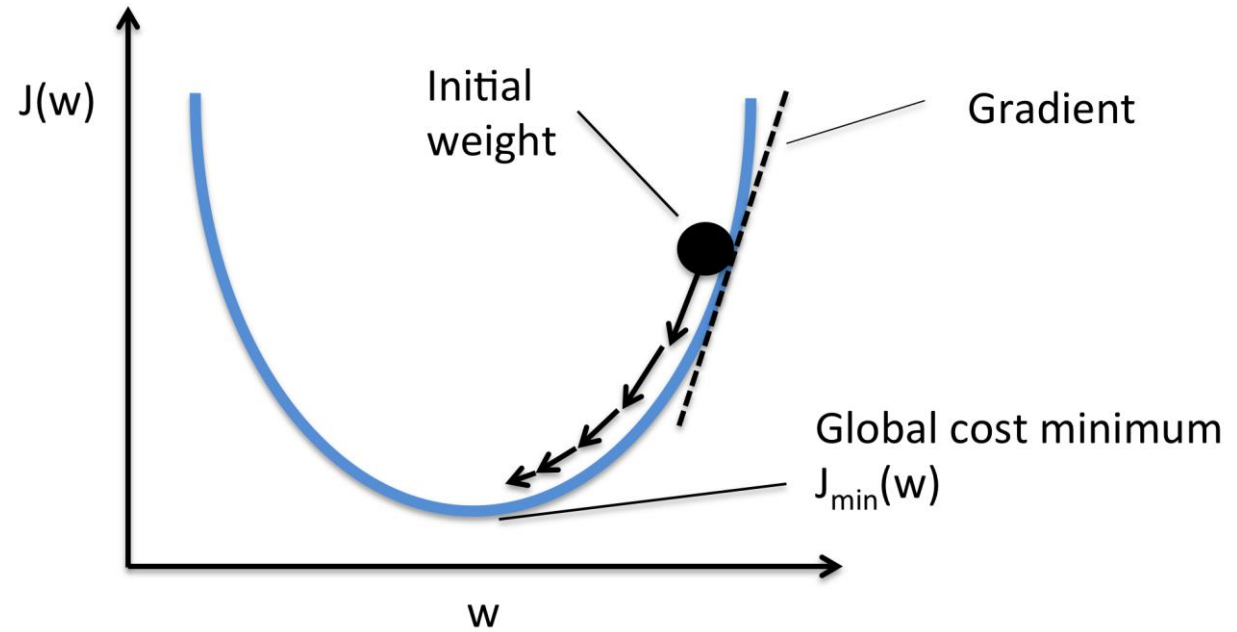
옵티마이저의 종류

Optimizers

02 옵티마이저의 종류 GD

GD Gradient Descent

- 함수의 기울기를 계산하여 그 반대방향으로 이동
- learning rate로 이동하는 거리 조절 가능
- 19세기 말 발견



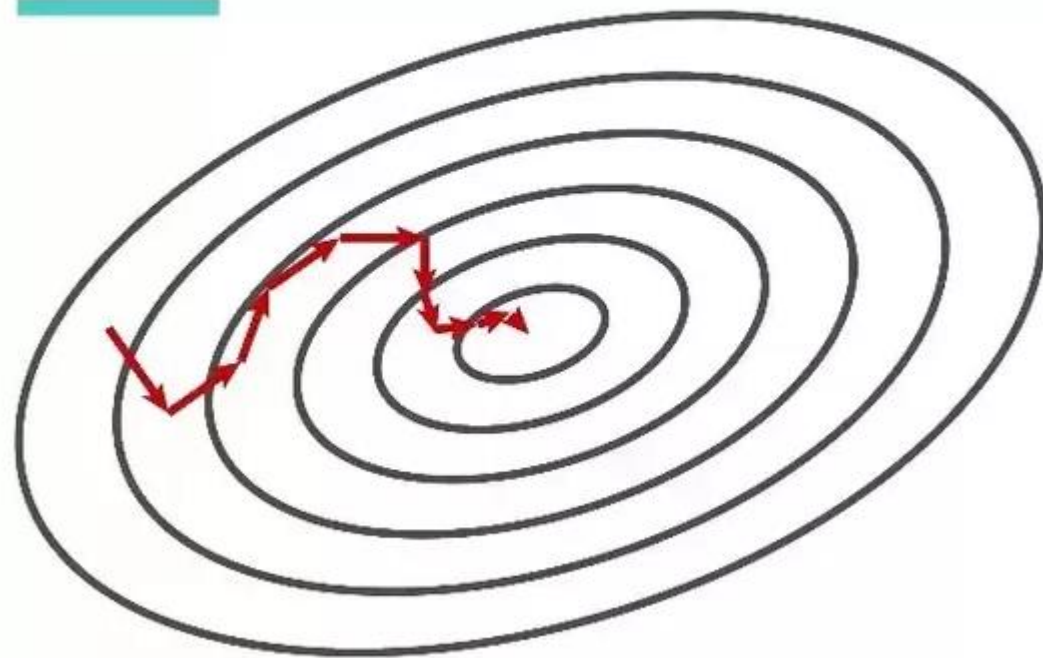
02 옵티마이저의 종류 SGD

SGD

Stochastic Gradient Descent

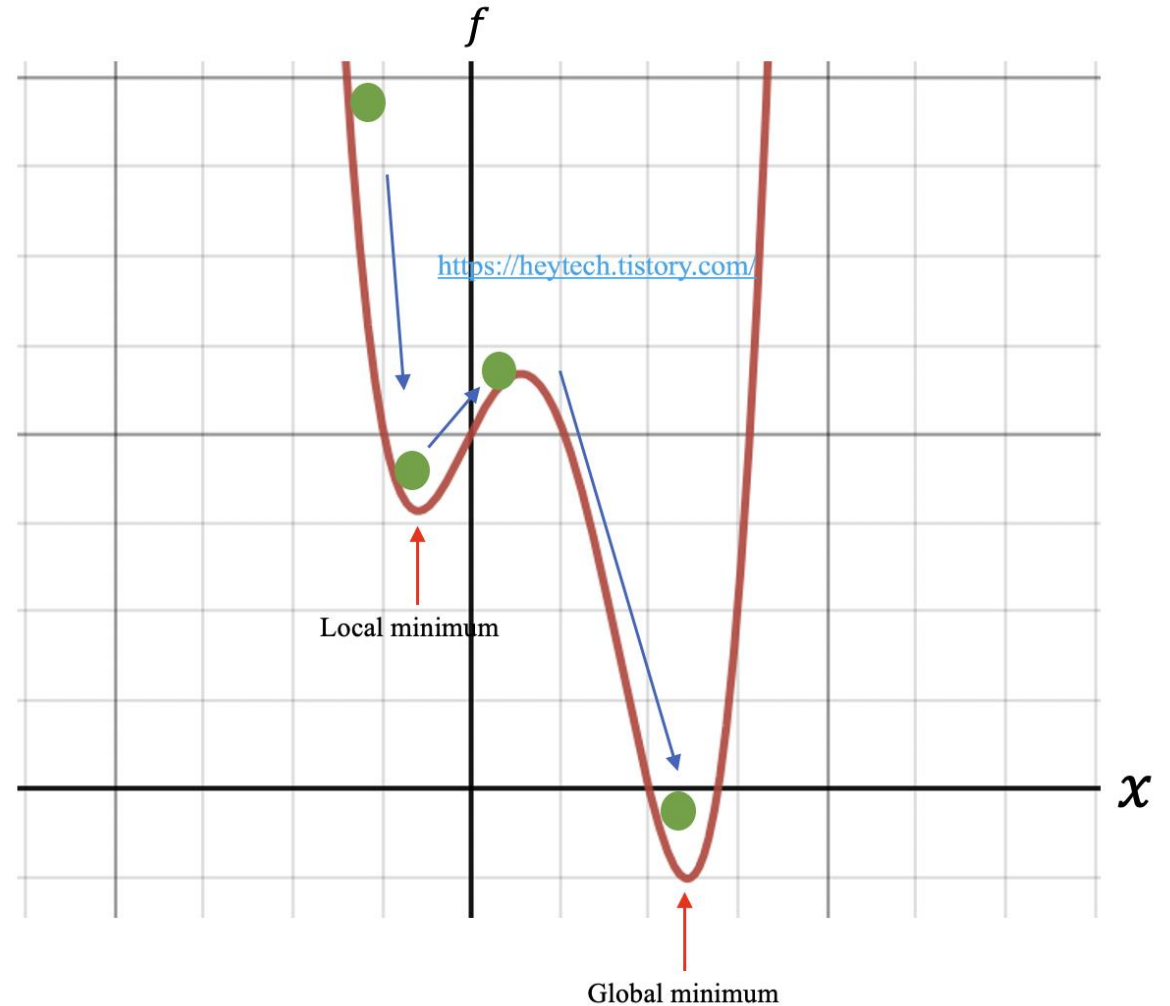
- 확률적 GD라는 의미
- 전체 데이터가 아닌 무작위로 선택된 샘플들에 대해 GD
- GD대비 Local Minima에 빠질 위험 적음
- 빠른 업데이트로 인해 대규모 데이터에서 효과적
- 최적화 과정 중 진동 발생 가능
- 1951년 제안됨

Stochastic Gradient Descent



Momentum

- 이전 step의 기울기가 현재 업데이트 정보에 영향을 끼침
- 실제 구슬을 굴리는 것 같이 관성을 가지게 됨
(그래서 이름이 모멘텀: 관성)
- 높은 수렴 속도와 적은 진동을 가짐
- 너무 큰 모멘텀은 발산 위험
- 안장점 회피 용이
- 1980년대에 제안됨

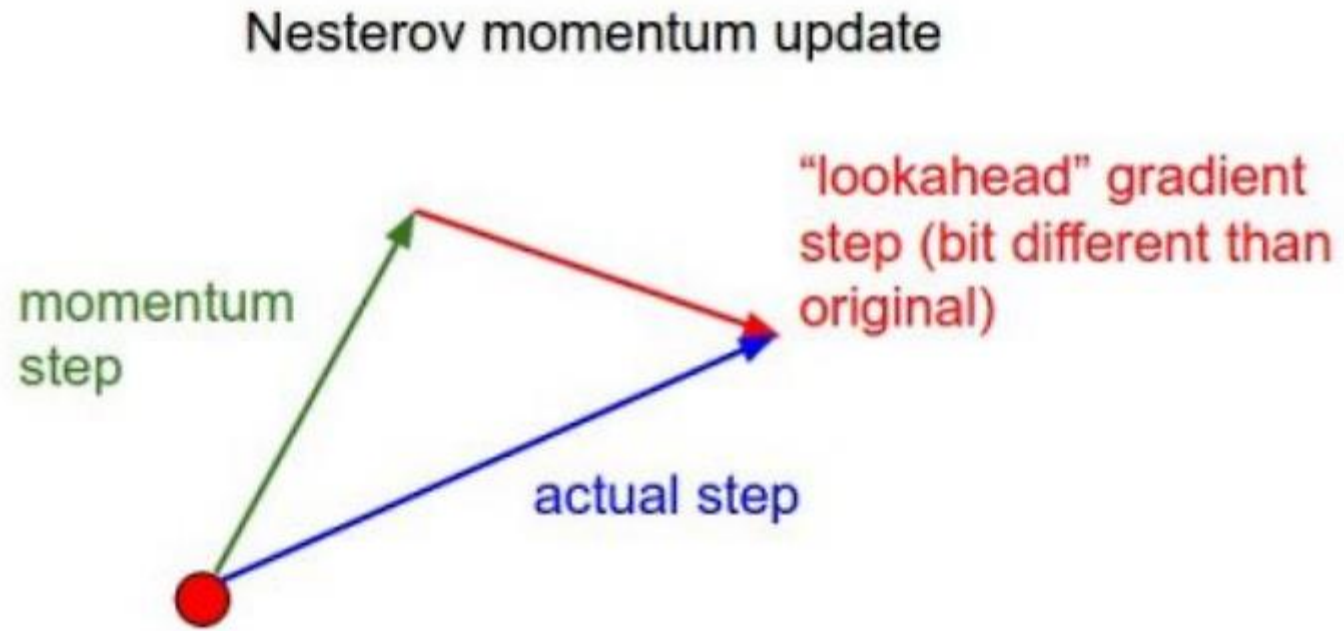


02 옵티마이저의 종류 NAG

NAG

Nesterov Accelerated Gradient

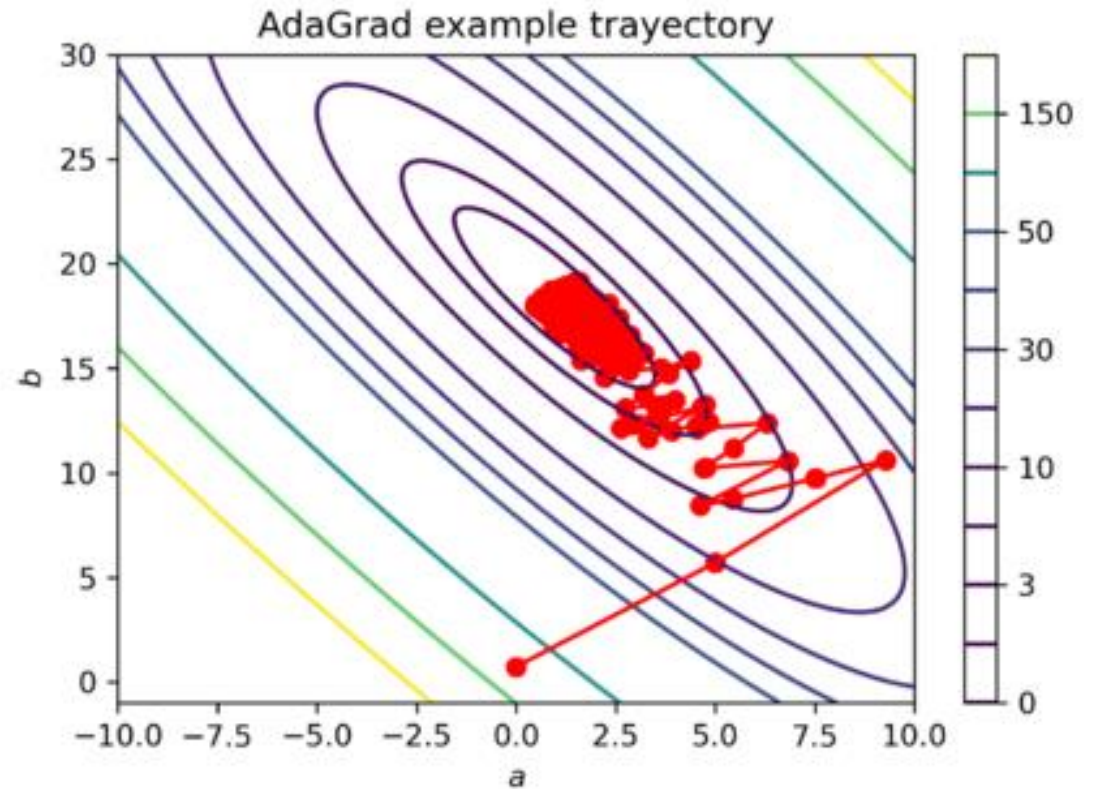
- 모멘텀을 개선한 방식
- 미리 계산된 위치에서 기울기를 구함
- 모멘텀보다 더 빠르고 정확
- 계산이 다소 복잡
- 1983년 제안됨



Adagrad

Adaptive Gradients

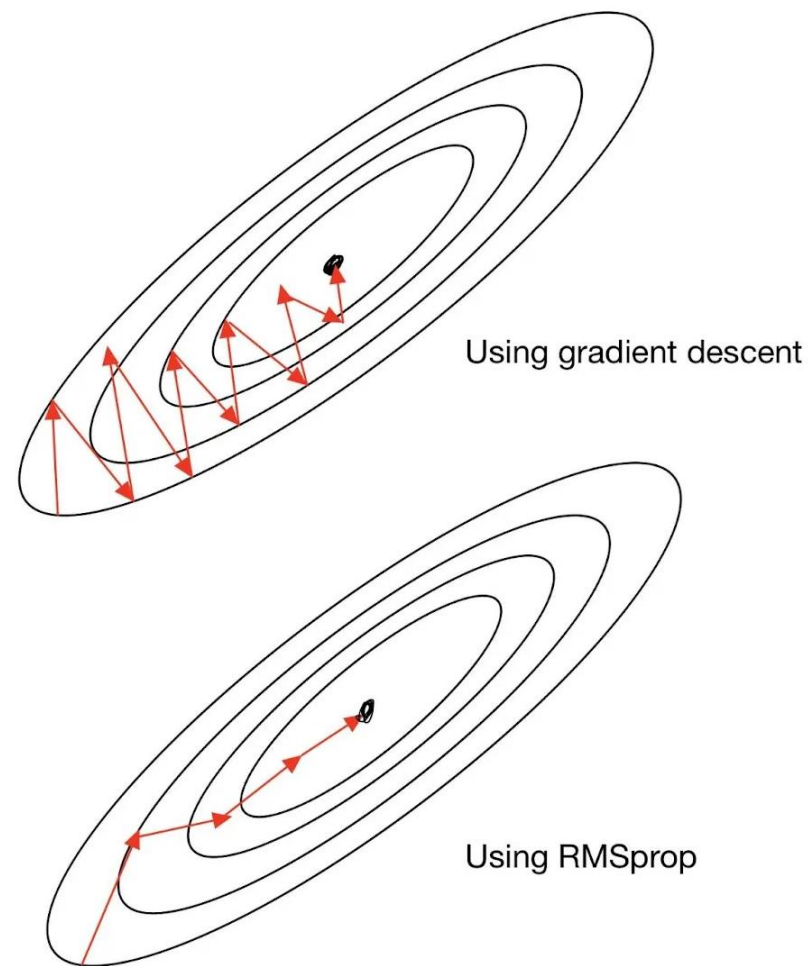
- 기울기가 급격한 곳은 빠르게 스킵하고, 완만한 곳은 천천히 이동
- 이를 위해 업데이트마다 Learning Rate 조절
- 희소한 특징 빠르게 학습 가능
- 희소한 데이터 처리, 자연어 처리 등에 사용됨
- 학습률이 너무 감소하면 나중에 학습이 느려질 수 있음
- 2011년 제안됨



RMSprop

Root Mean Squared Propagation

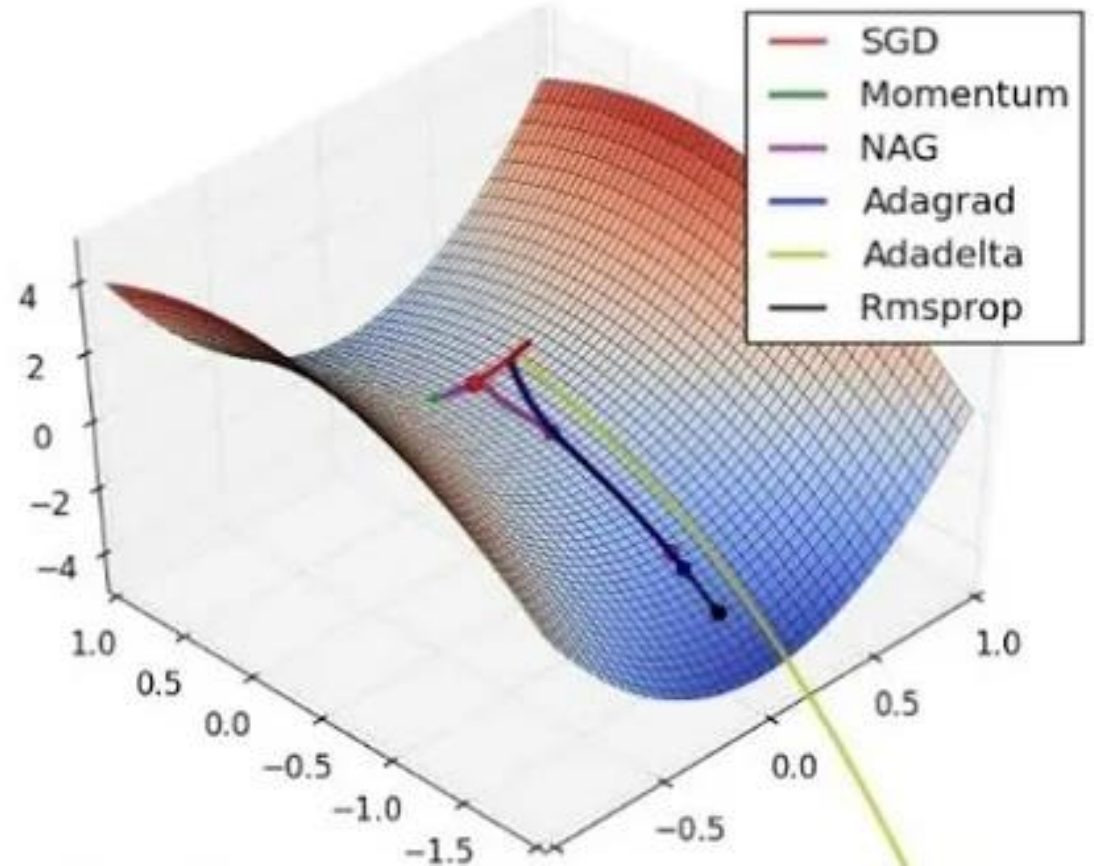
- Adagrad는 이전의 기울기 정보를 전부 반영하는 반면, RMSprop은 오래되면 오래될수록 덜 반영
- Adagrad 대비 학습률이 너무 작아지는 일이 적음
- RNN같은 시계열 데이터 처리에 효과적
- 초기에 적절한 학습률을 선택해야 효과적
- 2012년 제안됨



AdaDelta

Adaptive Delta

- Adagrad의 변형으로, 학습률을 국소적으로 조정하여 학습률이 지수적으로 감소하는 것 방지
- 적응형으로 학습률 조정
- Learning Rate에 대한 fine tuning 불필요
- 딥러닝 모델에서 효과적
- 복잡한 모델에서는 성능 하락 위험
- 2012년 제안됨

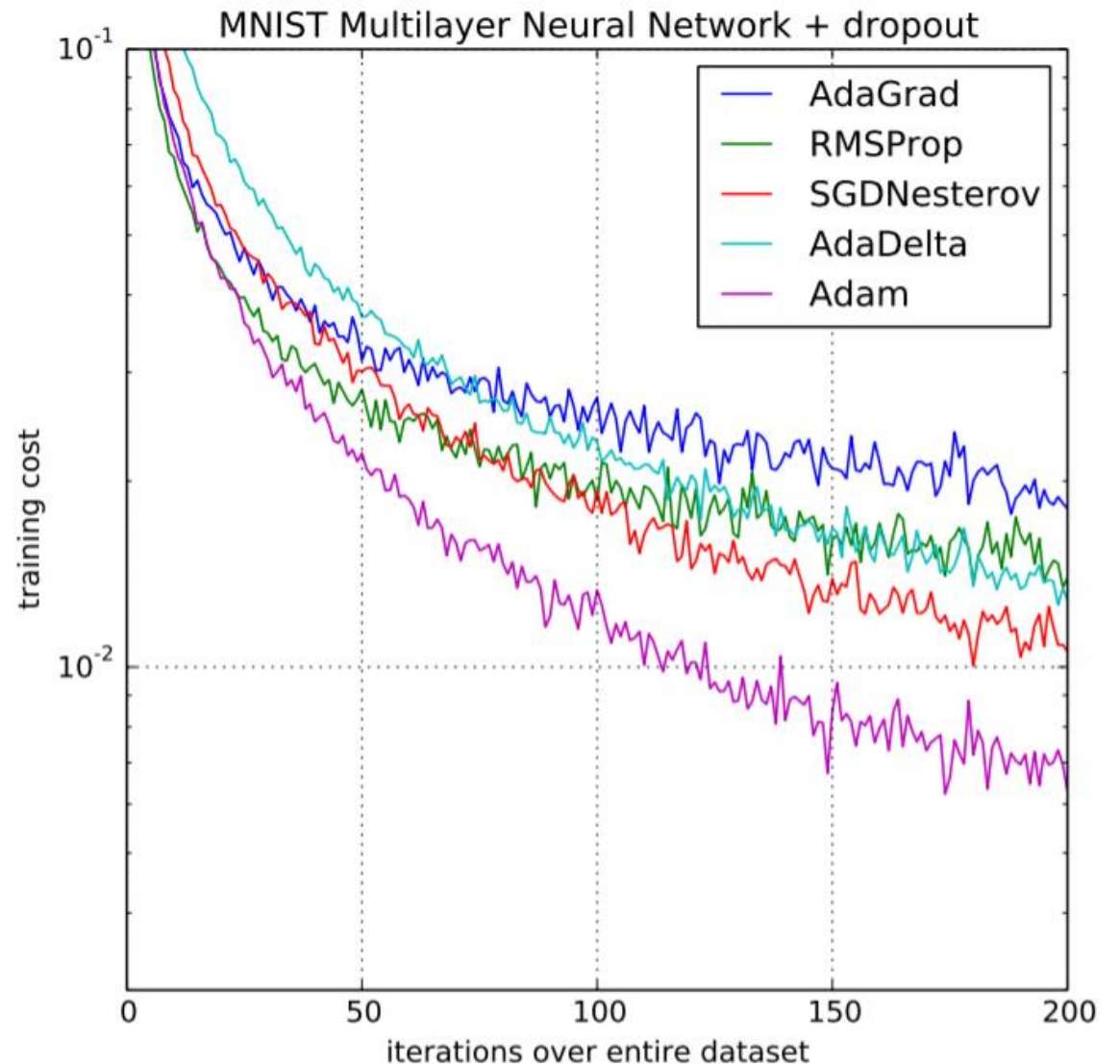


02 옵티마이저의 종류 Adam

Adam

Adaptive Moment Estimation

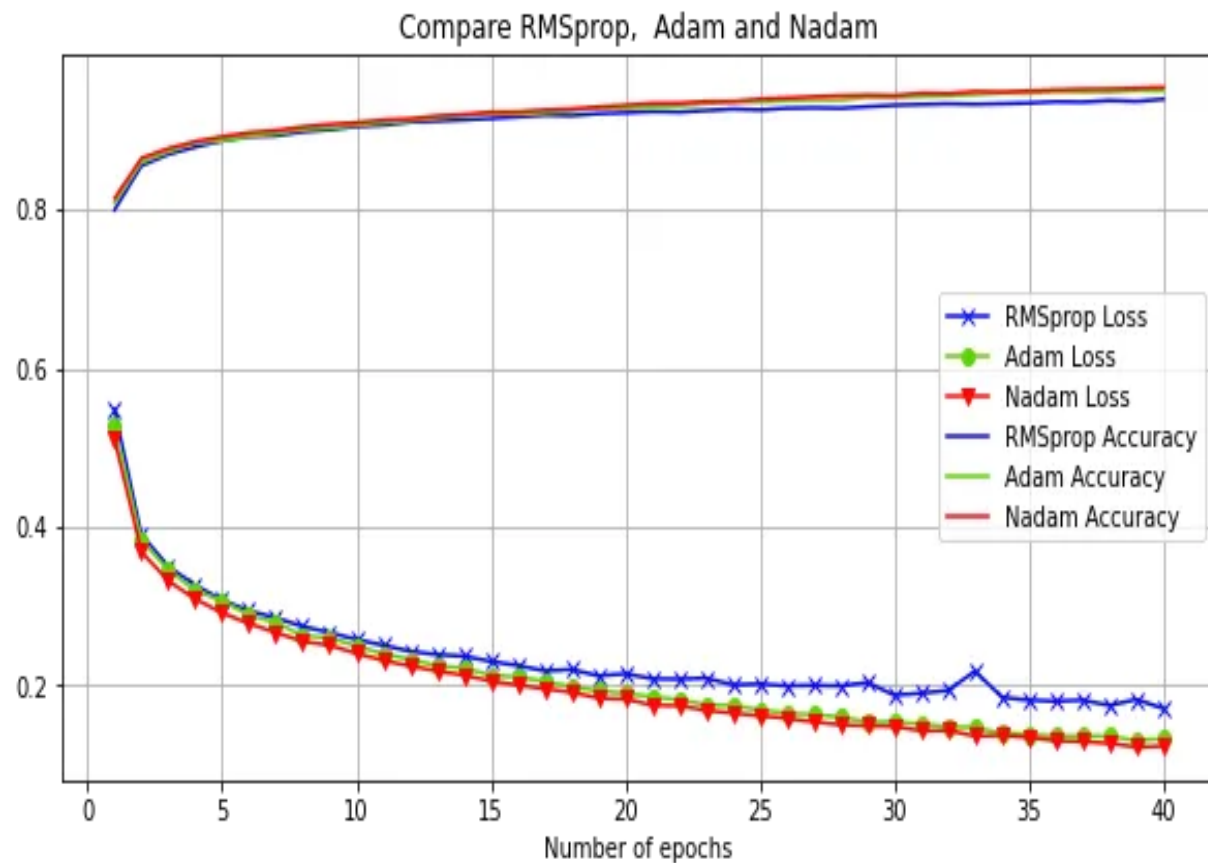
- RMSprop에 Momentum을 결합
- 각 파라미터에 대해 적응형 학습률과 모멘텀 동시 사용
- 대부분 딥러닝 모델에서 디폴트 옵티마이저로 사용
- 타 옵티마이저 대비 빠른 속도
- 고정된 학습률이 필요할 때는 성능 감소 위험
- 안장점 회피 용이
- 2014년 제안됨



Nadam

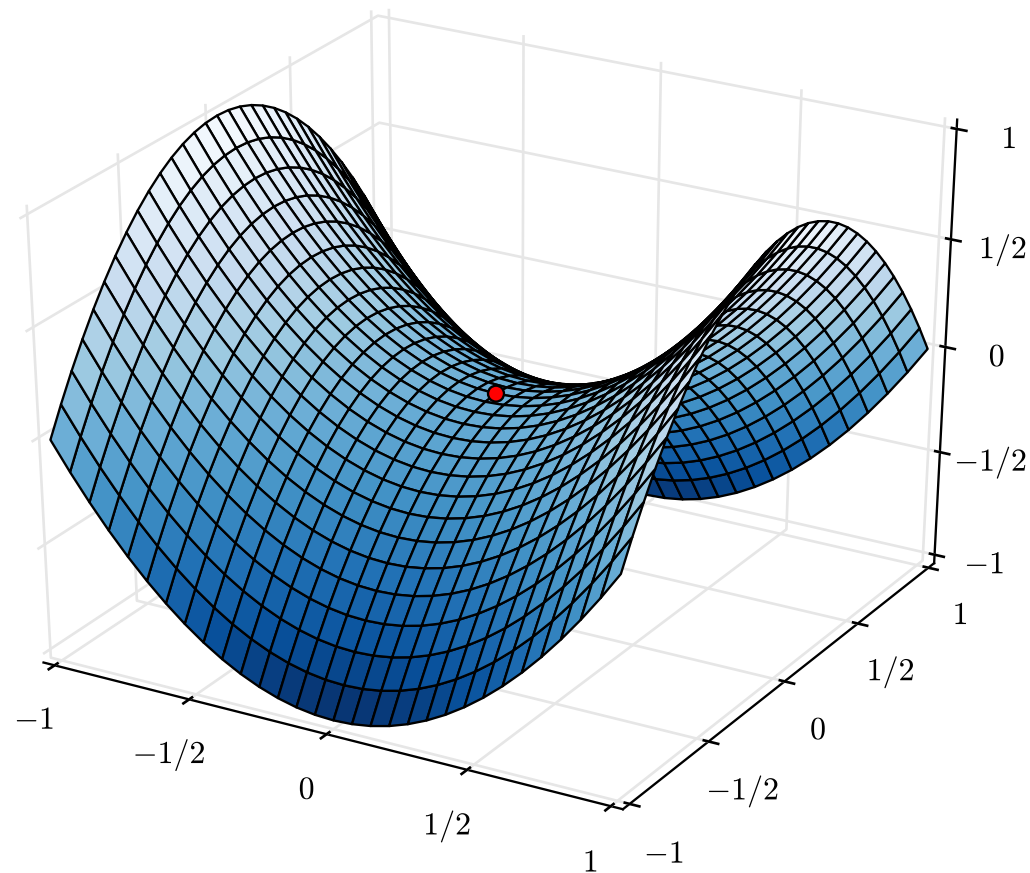
Nesterov Adaptive Moment Estimation

- Adam에 NAG을 결합
- 빠른 수렴이 필요한 딥러닝 모델에서 사용
- 조금 더 안정적이고 빠른 학습
- 특정 상황에서는 Adam보다 성능이 떨어질 수 있음
- 안장점 회피 용이
- 2016년 제안됨



안장점 Saddle Point

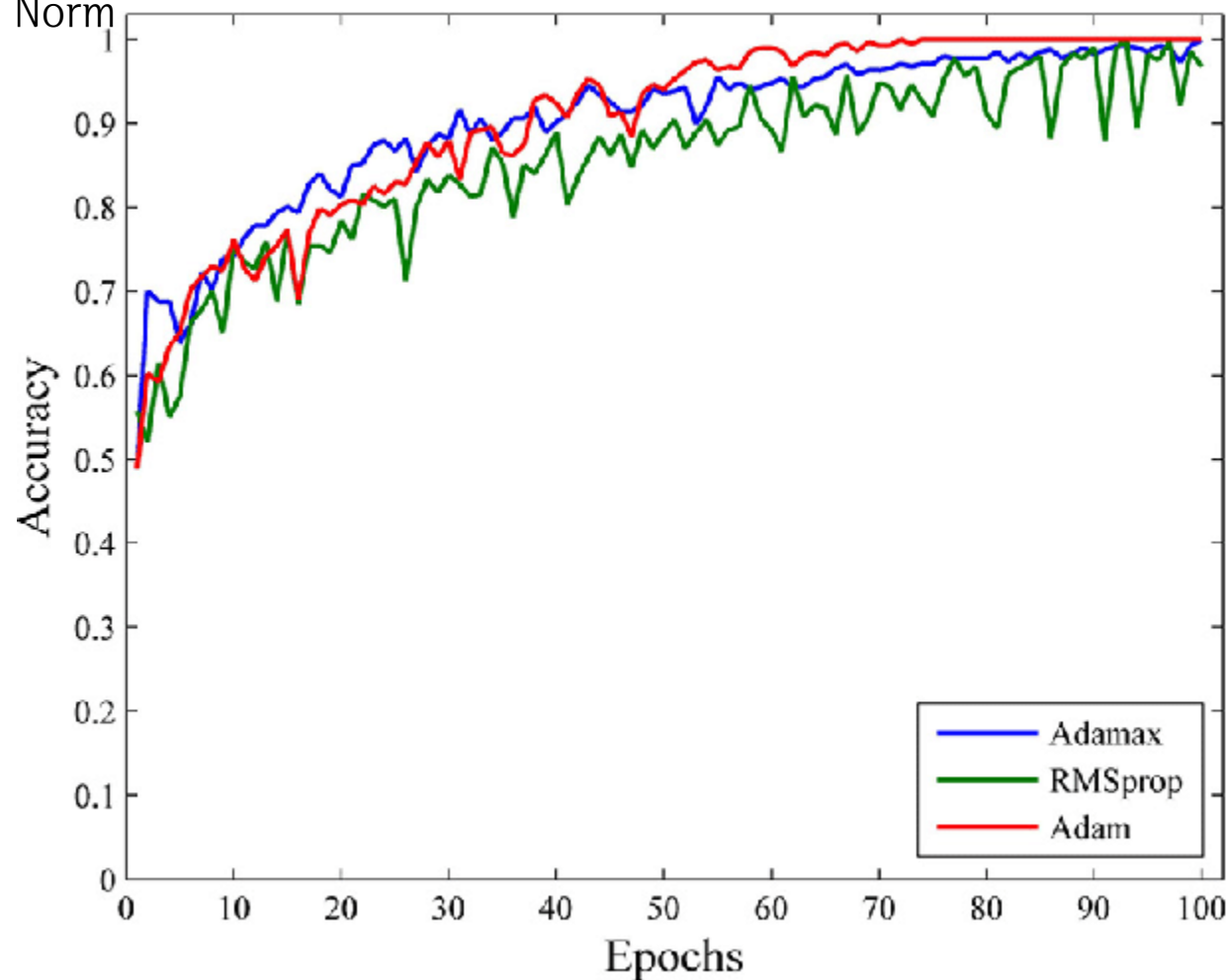
- 함수의 기울기가 0이지만, 극대나 극소가 아닌 지점
- 딥러닝 모델의 비용함수는 수많은 안장점 존재
- 안장점의 기울기는 0이므로 학습이 느려지거나 멈출 수 있음
- 안장점과 Local Minima를 어떻게 피하는가가 옵티마이저의 관건
- Momentum이나 Adam같은 옵티마이저들은 안장점을 잘 피할 수 있도록 설계됨



AdaMax

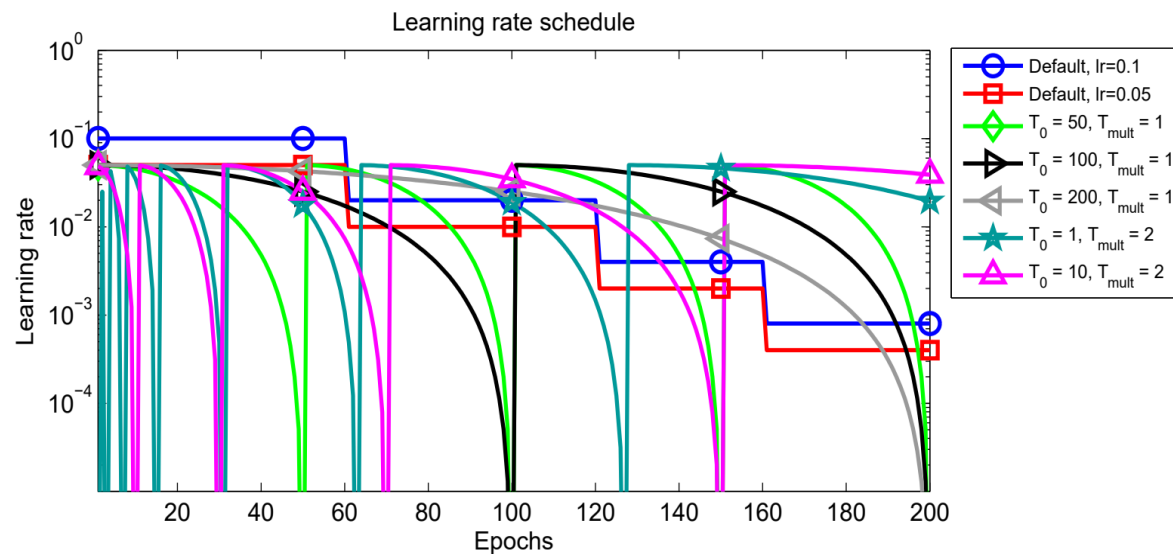
Adaptive Moment Estimation with Infinity Norm

- Adam에 기반
- L^∞ norm을 이용하여 더 안정적이고 효율적인 학습 가능
- Adam과 유사한 성능이나. 특정 상황에서만 더 우수
- 2015년 제안됨



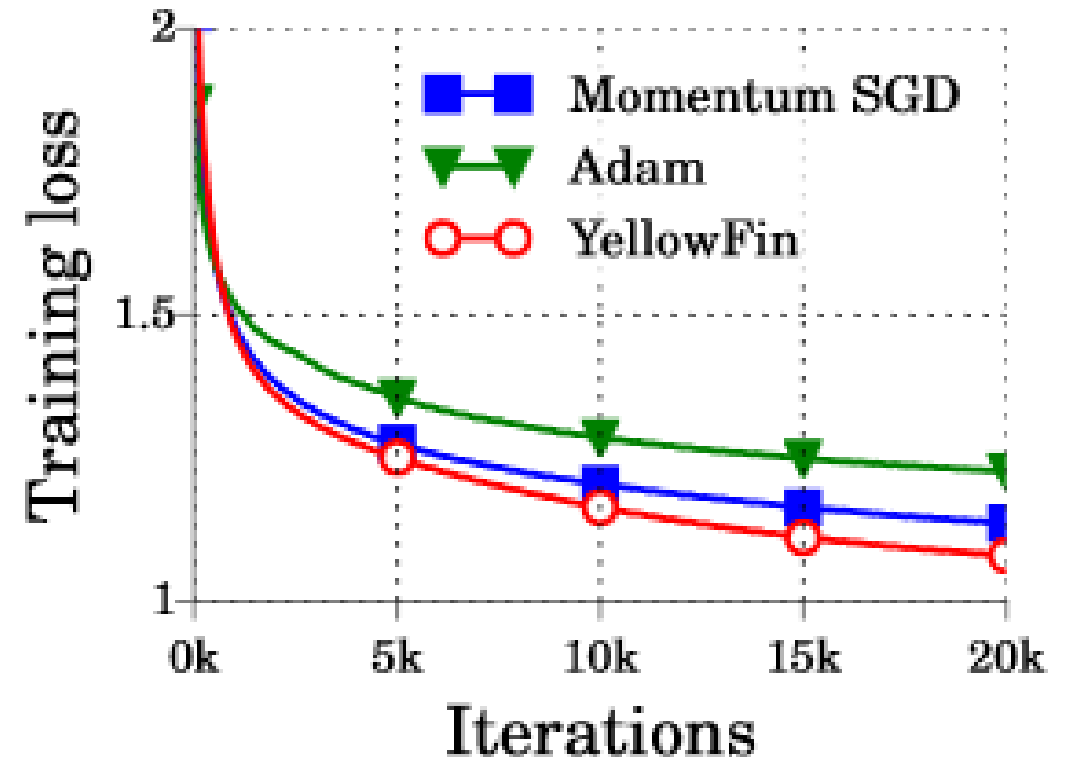
SGDR Stochastic Gradient Descent with Warm Restarts

- SGD에 기반
- 학습률을 주기적으로 감소시키고, 일정 주기 후에 다시 증가
- 빠른 수렴, 일반화 성능 높음
- 학습률 조정 주기의 설정 필요
- 2016년 제안됨
- [논문 링크 \(클릭\)](#)



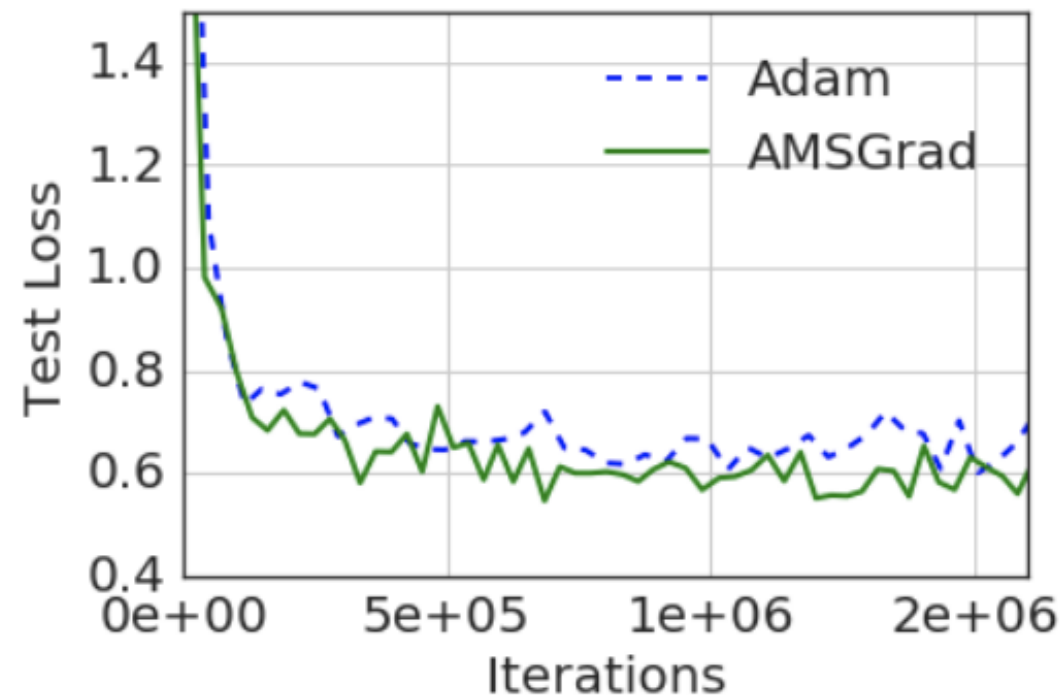
YellowFin

- 모멘텀과 학습률을 자동으로 조정
- 과도한 파인튜닝 없이 모델 학습 가능
- 자동 조정 과정에서 Overfitting 발생 위험
- 2017년 제안됨
- [논문 링크 \(클릭\)](#)



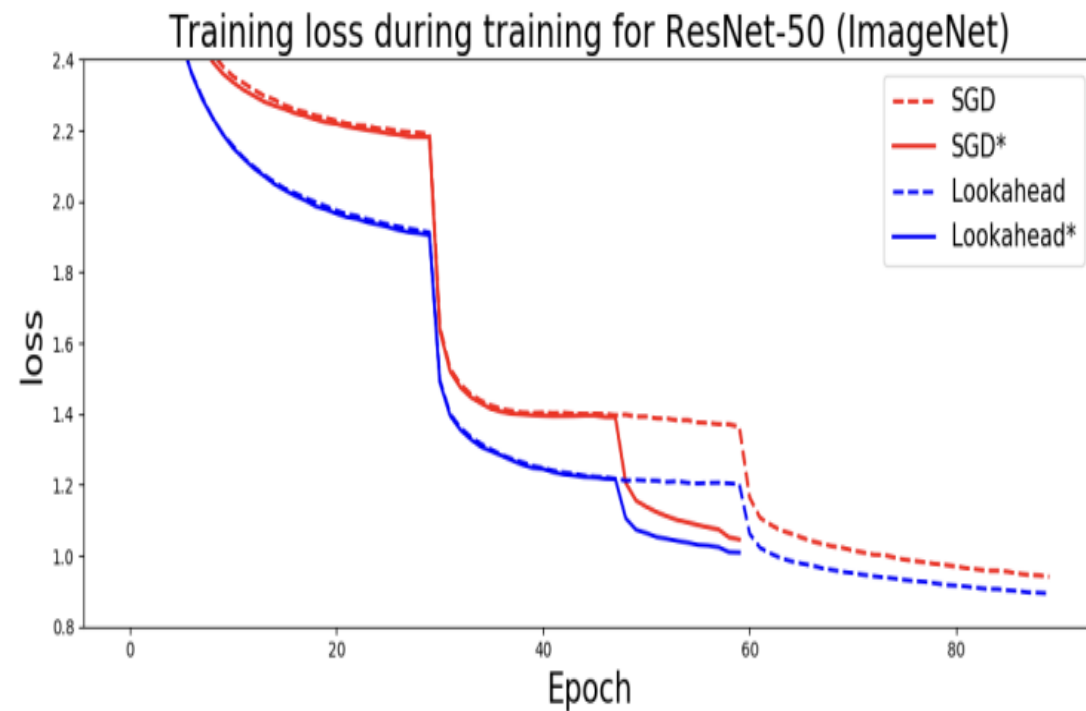
AMSgrad

- Adam에 기반
- Adam에서는 기울기의 2차 모멘트의 누적으로 수렴이 불안정해질 수 있는데, AMSgrad는 이 문제를 개선
- 더 나은 수렴과 안정성 제공
- 일부 문제에서는 Adam과 큰 차이가 없음
- 2019년 제안됨
- [논문 링크 \(클릭\)](#)



Lookahead

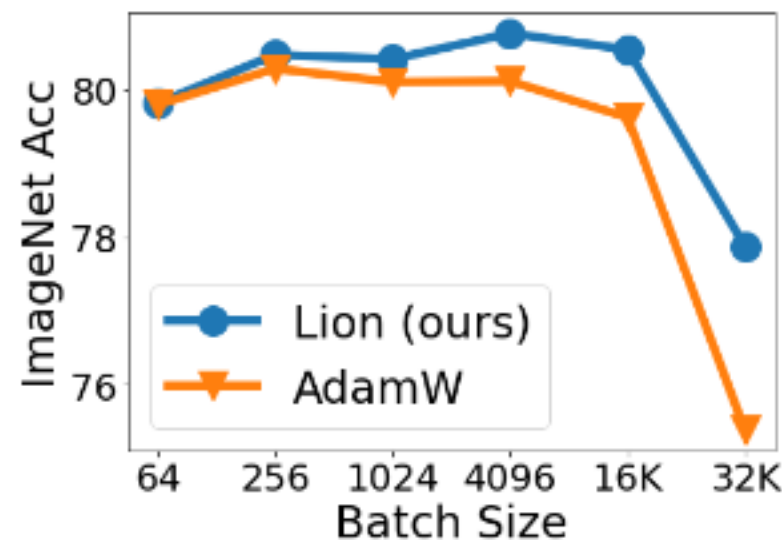
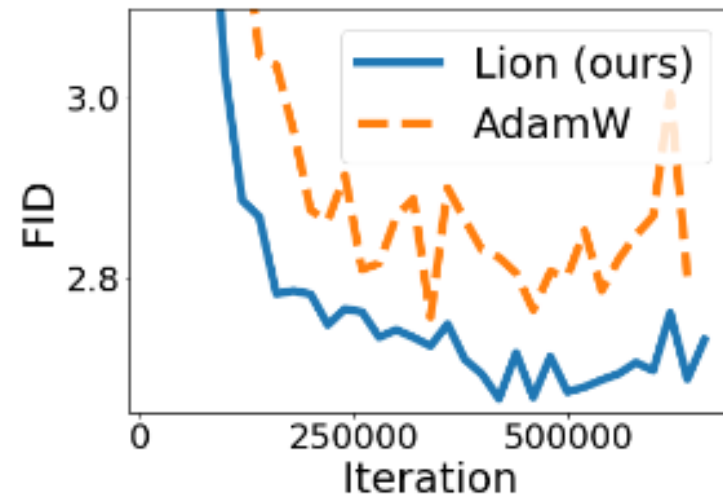
- 독립된 옵티마이저보다는 momentum처럼 기존 옵티마이저에 추가할 수 있는 확장팩 느낌
- k번의 업데이트를 미리 진행(fast weight) 후 최종 weight(slow weight)를 k번 업데이트된 weight를 따라가도록 업데이트
- 2019년 제안됨
- [논문 링크 \(클릭\)](#)



02 옵티마이저의 종류 기타 옵티마이저들

Lion EvoLved Sign Momentum

- Adam 대비 메모리 효율적
- 신경망에서 강력한 성능을 보여줌
- 기울기 대신 모멘텀만 추적, 쉬운 연산으로 업데이트 계산
- 2023년 제안됨
- [논문 링크 \(클릭\)](#)



03

실습 #1

DNN by keras

실습 #1)

keras를 활용하여 Deep Neural Network 구현

- 실습 예시에서 사용한 Dense 층을 활용하여 DNN 구현
- Chap.1에서 사용한 “NN_data.csv” 데이터 사용
- label 데이터는 One-Hot Encoding 하여 사용
- Train set, Test set을 분할하여 사용

03 실습 #1 코드 분석

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras
from keras.layers import Dense
from keras.models import Sequential
from keras.optimizers import SGD
```

필요한 모듈 임포트

```
def One_Hot_Encoding(y, number_of_class):
    """원 - 핫 인코딩을 실행하는 함수"""
    one_hot = np.zeros((len(y), number_of_class))
    if (min(y) == 0):
        for i in range(len(y)):
            one_hot[i, int(y[i])] = 1
    else:
        for i in range(len(y)):
            one_hot[i, int(y[i] - 1)] = 1
    return one_hot
```

필요한 함수들을 정의한다.

03 실습 #1 코드 분석

```
folder_location = "C:\\YS\\TUK\\Python_Class\\Advanced_Machine_Learning\\source\\NN_data.csv"
data = pd.read_csv(folder_location).drop(columns=['Unnamed: 0'])
X = data.iloc[:, :-1].values # 입력 데이터 (x0, x1, x2)
y = data.iloc[:, -1].values # 출력 데이터 (클래스)
num_classes = len(np.unique(y)) # 클래스 수
y_one_hot = One_Hot_Encoding(y, num_classes)
```

NN_data.csv를 불러오고, 전처리 시행
이 때, y값은 one-hot encoding해 둔다.

```
indices = np.random.permutation(X.shape[0]) # 인덱스를 무작위로 섞기
train_size = int(0.7 * len(indices))
val_size = int(0.1 * len(indices))
test_size = len(indices) - train_size - val_size
train_indices = indices[:train_size]
val_indices = indices[train_size:train_size + val_size]
test_indices = indices[train_size + val_size:]
X_train, X_val, X_test = X[train_indices], X[val_indices], X[test_indices]
y_train, y_val, y_test = y_one_hot[train_indices], y_one_hot[val_indices], y_one_hot[test_indices]
```

Training Set : Validation Set : Test Set = 7 : 1 : 2로 나누는 과정

03 실습 #1 코드 분석

```
epochs = 100
batch_size = 32

model = Sequential([
    Dense(units=128, activation='sigmoid', input_shape=(X_train.shape[1],)),
    Dense(units=128, activation='sigmoid'),
    Dense(units=128, activation='sigmoid'),
    Dense(units=64, activation='sigmoid'),
    Dense(units=num_classes, activation='softmax'),
])
```

Hyper Parameter 및 모델을 정의

모델은 128-128-128-64의 구조이며, 활성화함수로는 시그모이드, 출력층에서는 소프트맥스를 사용하였다.

모델 컴파일 및 학습

```
optimizer = SGD(learning_rate=0.3)
model.compile(loss='categorical_crossentropy', optimizer=optimizer, metrics=['accuracy'])
history = model.fit(X_train, y_train, epochs=epochs, batch_size=batch_size, validation_data=(X_val, y_val))
```

손실 및 정확도 기록 저장

```
Loss_history = history.history['loss']
Accuracy_history = history.history['accuracy']
Val_Loss_history = history.history['val_loss']
Val_Accuracy_history = history.history['val_accuracy']
```

정의된 모델을 컴파일 및 학습하고, Loss와 정확도를 기록하는 부분이다.

옵티마이저로는 SGD를 사용하였다.

03 실습 #1 코드 분석

```
# 그래프 그리기
epochs_range = range(epochs)
plt.figure(figsize=(10, 6))

# Loss와 Validation Loss
plt.plot(epochs_range, Loss_history, color='r', label='Loss')
plt.plot(epochs_range, Val_Loss_history, color='g', label='Validation Loss')

# Accuracy와 Validation Accuracy
plt.plot(epochs_range, Accuracy_history, color='b', label='Accuracy')
plt.plot(epochs_range, Val_Accuracy_history, color='y', label='Validation Accuracy')

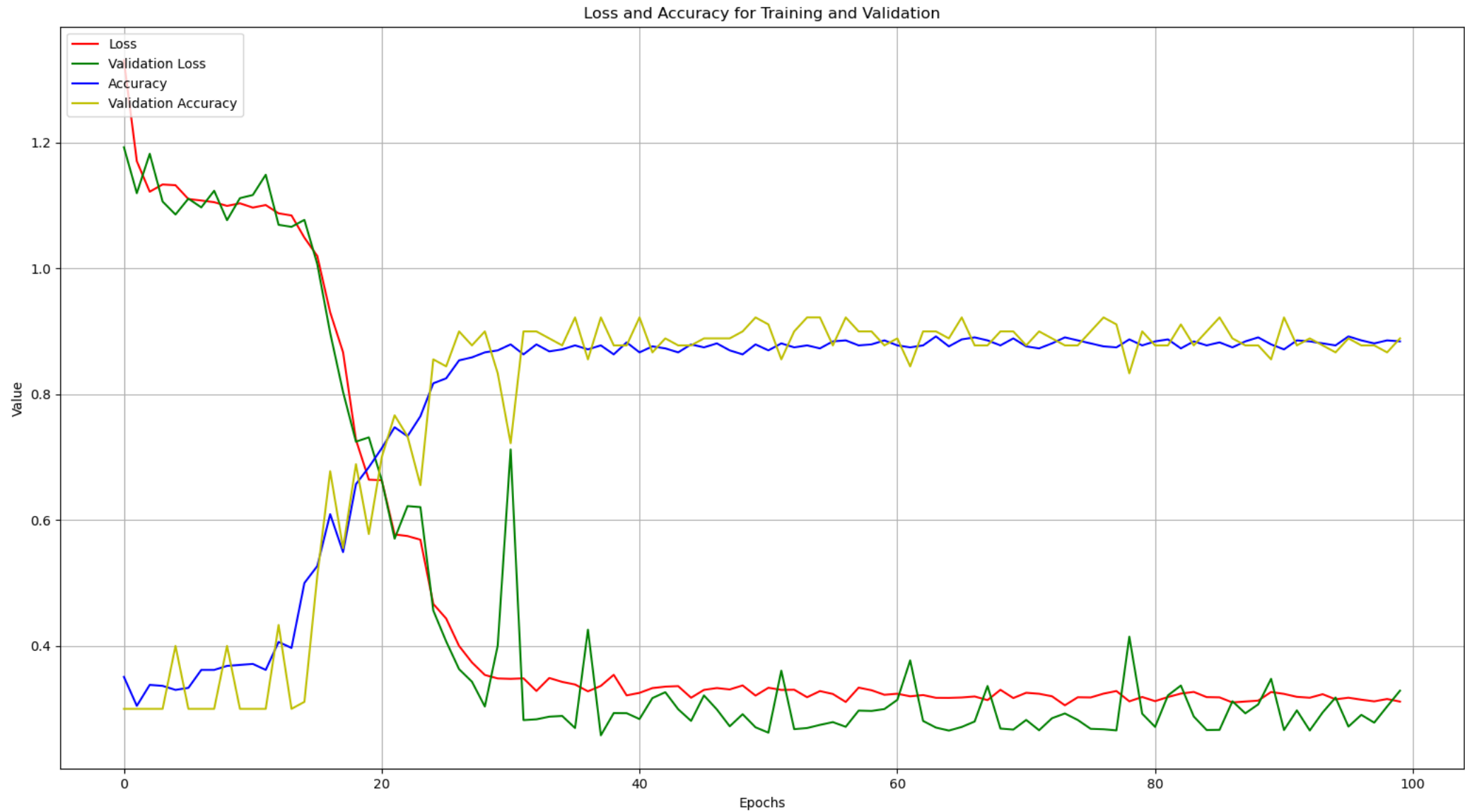
# 그래프 설정
plt.xlabel('Epochs')
plt.ylabel('Value')
plt.title('Loss and Accuracy for Training and Validation')
plt.legend(loc='upper left')
plt.grid()

# 테스트 세트로 모델 평가
test_loss, test_accuracy = model.evaluate(X_test, y_test)
val_loss, val_accuracy = model.evaluate(X_val, y_val)
print(f'\nTest Loss: {test_loss:.4f}\nTest Accuracy: {test_accuracy:.4f}\nValidation Loss: {val_loss:.4f}\nValidation Accuracy: {val_accuracy:.4f}')

plt.show()
```

그래프를 그리는 부분이다

03 실습 #1 실행 결과



04

실습 #3

Optimizer Comparison

실습 #2) Optimizer 비교 실습

- 실습 #1)에서 여러 Optimizer를 적용
- 각 Optimizer 마다 학습이 잘 되도록 최적의 하이퍼 파라미터 조정(활성화 함수, lr 등등)
- 각 Optimizer 별 학습 결과 비교 분석
- “NN_data”에 가장 최적인 Optimizer가 무엇인지 판별 및 이유 분석

규칙 Rules

- 옵티마이저는 총 5개, SGD, RMSprop, Adam, Adagrad, Lion을 사용한다.
- 각 옵티마이저마다 1개씩, 총 5개의 모델을 정의한다.
- Learning Rate와 Activation Function은 모든 모델이 동일하도록 한다.
- “최적”의 기준은 정확도로 한다.

04 실습 #2 코드 분석

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tensorflow import keras
from keras.layers import Dense
from keras.models import Sequential
from keras.optimizers import SGD
```

필요한 모듈 임포트

```
def One_Hot_Encoding(y, number_of_class):
    """원 - 핫 인코딩을 실행하는 함수"""
    one_hot = np.zeros((len(y), number_of_class))
    if (min(y) == 0):
        for i in range(len(y)):
            one_hot[i, int(y[i])] = 1
    else:
        for i in range(len(y)):
            one_hot[i, int(y[i] - 1)] = 1
    return one_hot
```

필요한 함수들을 정의한다.

04 실습 #2 코드 분석

```
folder_location = "C:\\YS\\TUK\\Python_Class\\Advanced_Machine_Learning\\source\\NN_data.csv"
data = pd.read_csv(folder_location).drop(columns=['Unnamed: 0'])
X = data.iloc[:, :-1].values # 입력 데이터 (x0, x1, x2)
y = data.iloc[:, -1].values # 출력 데이터 (클래스)
num_classes = len(np.unique(y)) # 클래스 수
y_one_hot = One_Hot_Encoding(y, num_classes)
```

NN_data.csv를 불러오고, 전처리 시행
이 때, y값은 one-hot encoding해 둔다.

```
indices = np.random.permutation(X.shape[0]) # 인덱스를 무작위로 섞기
train_size = int(0.8 * len(indices))
train_indices = indices[:train_size]
test_indices = indices[train_size:]
X_train, X_test = X[train_indices], X[test_indices]
y_train, y_test = y_one_hot[train_indices], y_one_hot[test_indices]
```

Training Set : Test Set = 8 : 2로 나누는 과정

```
epochs = 15
batch_size = 64
learning_rate = 0.001
activation_function = 'relu'
```

Hyper Parameter 및 활성화함수를 정의한다.

04 실습 #2 코드 분석

```
model_SGD = Sequential([
    Dense(units=128, activation=activation_function, input_shape=(X_train.shape[1],)),
    Dense(units=128, activation=activation_function),
    Dense(units=128, activation=activation_function),
    Dense(units=64, activation=activation_function),
    Dense(units=num_classes, activation='softmax'),
])
```

모델을 정의한다.

해당 모델은 옵티마이저로 SGD를 사용하기 위한 모델이며, 출력층의 활성화함수는 소프트맥스로 정의하였다.

```
model_RMSprop = Sequential([...
model_Adam = Sequential([...
model_Adagrad = Sequential([...
model_Lion = Sequential([...
```

나머지 4개의 모델도 정의해준다.

각각의 모델은 이름만 다르고 내부 구조는 완전히 동일하다.

옵티마이저만을 비교하기 위함이다.

04 실습 #2 코드 분석

```
start_time = t.time()
optimizer = SGD(learning_rate=learning_rate)
model_SGD.compile(loss='categorical_crossentropy', optimizer=optimizer, metrics=['accuracy'])
history_SGD = model_SGD.fit(X_train, y_train, epochs=epochs, batch_size=batch_size)
end_time = t.time()
training_time['SGD'] = end_time - start_time
print("SGD 모델 학습 완료!\n")
```

모델을 컴파일하고 학습시킨다.

오차로는 CEE를 사용하였으며, 현재 epoch은 15, learning rate는 0.01, batch size = 64이다.

```
start_time = t.time()
optimizer = RMSprop(learning_rate=learning_rate)
model_RMSprop.compile(loss='categorical_crossentropy', optimizer=optimizer, metrics=['accuracy'])
history_RMSprop = model_RMSprop.fit(X_train, y_train, epochs=epochs, batch_size=batch_size)
end_time = t.time()
training_time['RMSprop'] = end_time - start_time
print("RMSprop 모델 학습 완료!\n")
```

```
start_time = t.time()
optimizer = Adam(learning_rate=learning_rate)
model_Adam.compile(loss='categorical_crossentropy', optimizer=optimizer, metrics=['accuracy'])
history_Adam = model_Adam.fit(X_train, y_train, epochs=epochs, batch_size=batch_size)
end_time = t.time()
training_time['Adam'] = end_time - start_time
print("Adam 모델 학습 완료!\n")
```

```
start_time = t.time()...
```

나머지 4개의 모델에 대해서도 동일한 작업을 해준다.

04 실습 #2 코드 분석

```
Loss_history_SGD = history_SGD.history['loss']
Accuracy_history_SGD = history_SGD.history['accuracy']
Loss_history_RMSprop = history_RMSprop.history['loss']
Accuracy_history_RMSprop = history_RMSprop.history['accuracy']
Loss_history_Adam = history_Adam.history['loss']
Accuracy_history_Adam = history_Adam.history['accuracy']
Loss_history_Adagrad = history_Adagrad.history['loss']
Accuracy_history_Adagrad = history_Adagrad.history['accuracy']
Loss_history_Lion = history_Lion.history['loss']
Accuracy_history_Lion = history_Lion.history['accuracy']
```

학습된 모델의 손실과 정확도를 기록하여 저장한다.

04 실습 #2 코드 분석

```
epochs_range = range(epochs)
colors = ['r', 'g', 'b', 'y', 'm', 'c', 'orange', 'purple', 'pink', 'brown']

plt.subplot(2,1,1)
plt.plot(epochs_range, Loss_history_SGD, color=colors[0], label='Loss (SGD)')
plt.plot(epochs_range, Loss_history_RMSprop, color=colors[1], label='Loss (RMSprop)')
plt.plot(epochs_range, Loss_history_Adam, color=colors[2], label='Loss (Adam)')
plt.plot(epochs_range, Loss_history_Adagrad, color=colors[3], label='Loss (Adagrad)')
plt.plot(epochs_range, Loss_history_Lion, color=colors[4], label='Loss (Lion)')
plt.title(f'Final Loss for Different Optimizers, Activation: {activation_function}')
plt.grid()
final_loss = [
    f'SGD Loss: {Loss_history_SGD[-1]:.4f}',...
]
plt.legend(final_loss, loc='upper right')

plt.subplot(2,1,2)
plt.plot(epochs_range, Accuracy_history_SGD, color=colors[0], label='Accuracy (SGD)')
plt.plot(epochs_range, Accuracy_history_RMSprop, color=colors[1], label='Accuracy (RMSprop)')
plt.plot(epochs_range, Accuracy_history_Adam, color=colors[2], label='Accuracy (Adam)')
plt.plot(epochs_range, Accuracy_history_Adagrad, color=colors[3], label='Accuracy (Adagrad)')
plt.plot(epochs_range, Accuracy_history_Lion, color=colors[4], label='Accuracy (Lion)')
plt.title(f'Final Accuracy for Different Optimizers, Activation: {activation_function}')
plt.grid()
final_accuracies = [
    f'SGD Accuracy: {Accuracy_history_SGD[-1]:.4f}',...
]
plt.legend(final_accuracies, loc='lower right')
```

두 개의 그래프 중 오차와 정확도를 그리는 부분이다.

04 실습 #2 코드 분석

```
plt.figure(2)
plt.grid(zorder=0)
bars = plt.bar(list(training_time.keys()), list(training_time.values()), color=['r', 'g', 'b', 'y', 'm'], zorder=5)
plt.title('Training Time for Different Optimizers', fontsize=14)
plt.xlabel('Optimizer', fontsize=12)
plt.ylabel('Training Time (seconds)', fontsize=12)
for values in bars:
    height = values.get_height()
    plt.text(values.get_x() + values.get_width()/2, height + 0.03, f'{height:.2f}', fontsize=10)
```

두 개의 그래프 중 각 모델별 소요 시간을 나타내는 부분이다.

```
test_loss_SGD, test_accuracy_SGD = model_SGD.evaluate(X_test, y_test)
test_loss_RMSprop, test_accuracy_RMSprop = model_RMSprop.evaluate(X_test, y_test)
test_loss_Adam, test_accuracy_Adam = model_Adam.evaluate(X_test, y_test)
test_loss_Adagrad, test_accuracy_Adagrad = model_Adagrad.evaluate(X_test, y_test)
test_loss_Lion, test_accuracy_Lion = model_Lion.evaluate(X_test, y_test)

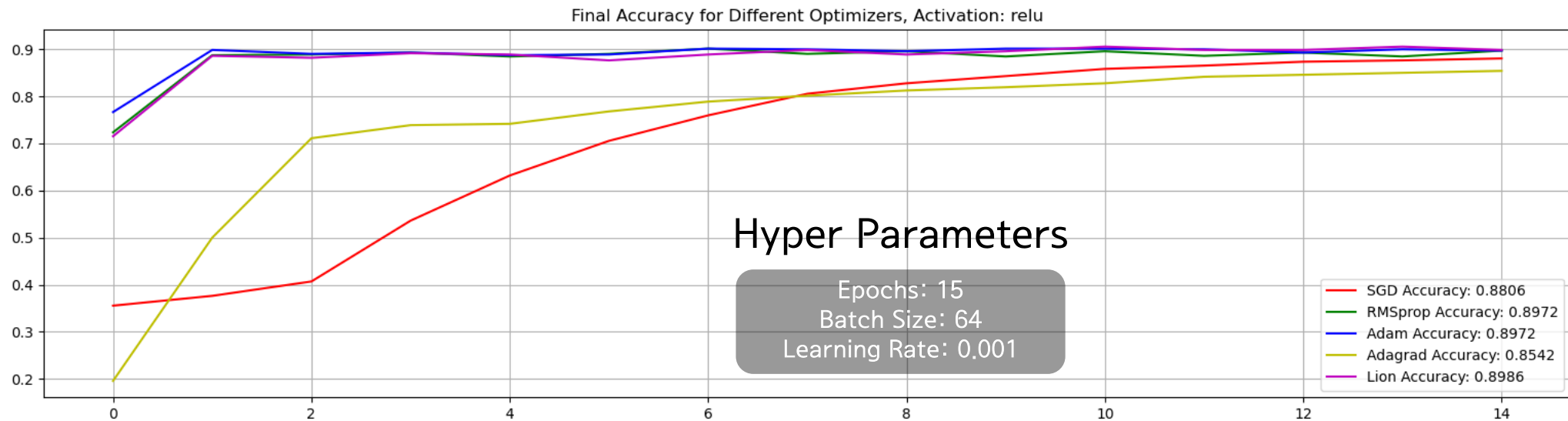
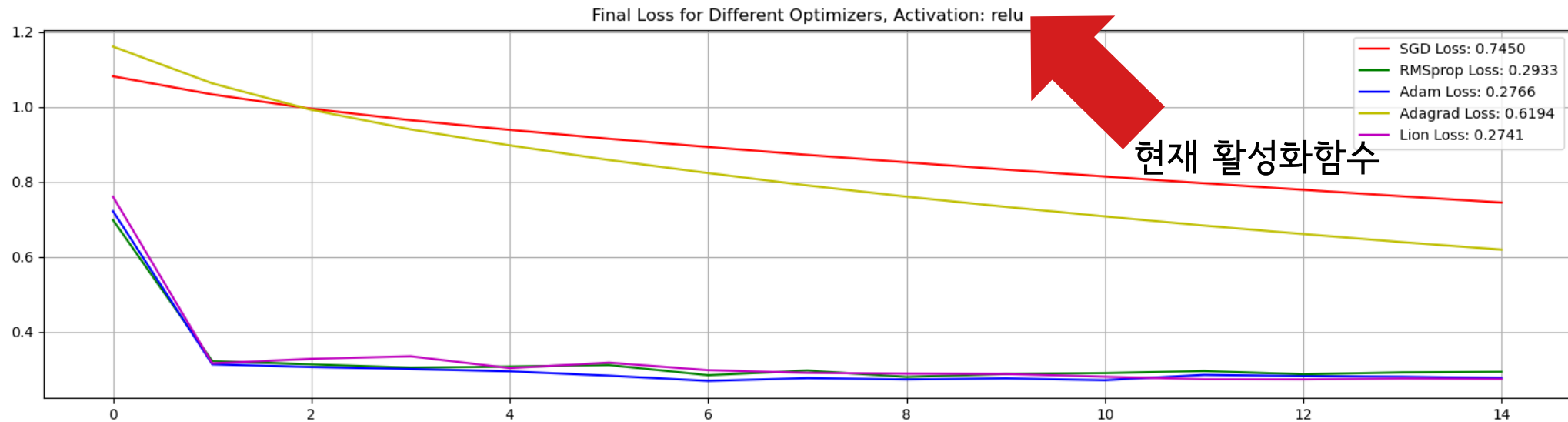
print(f'\nTest Loss (SGD): {test_loss_SGD:.4f}\nTest Accuracy (SGD): {test_accuracy_SGD:.4f}')
print(f'\nTest Loss (RMSprop): {test_loss_RMSprop:.4f}\nTest Accuracy (RMSprop): {test_accuracy_RMSprop:.4f}')
print(f'\nTest Loss (Adam): {test_loss_Adam:.4f}\nTest Accuracy (Adam): {test_accuracy_Adam:.4f}\n')
print(f'\nTest Loss (Adagrad): {test_loss_Adagrad:.4f}\nTest Accuracy (Adagrad): {test_accuracy_Adagrad:.4f}\n')
print(f'\nTest Loss (Lion): {test_loss_Lion:.4f}\nTest Accuracy (Lion): {test_accuracy_Lion:.4f}\n')

for model_name, duration in training_time.items():
    print(f'Training time for {model_name}: {duration:.2f} 초')

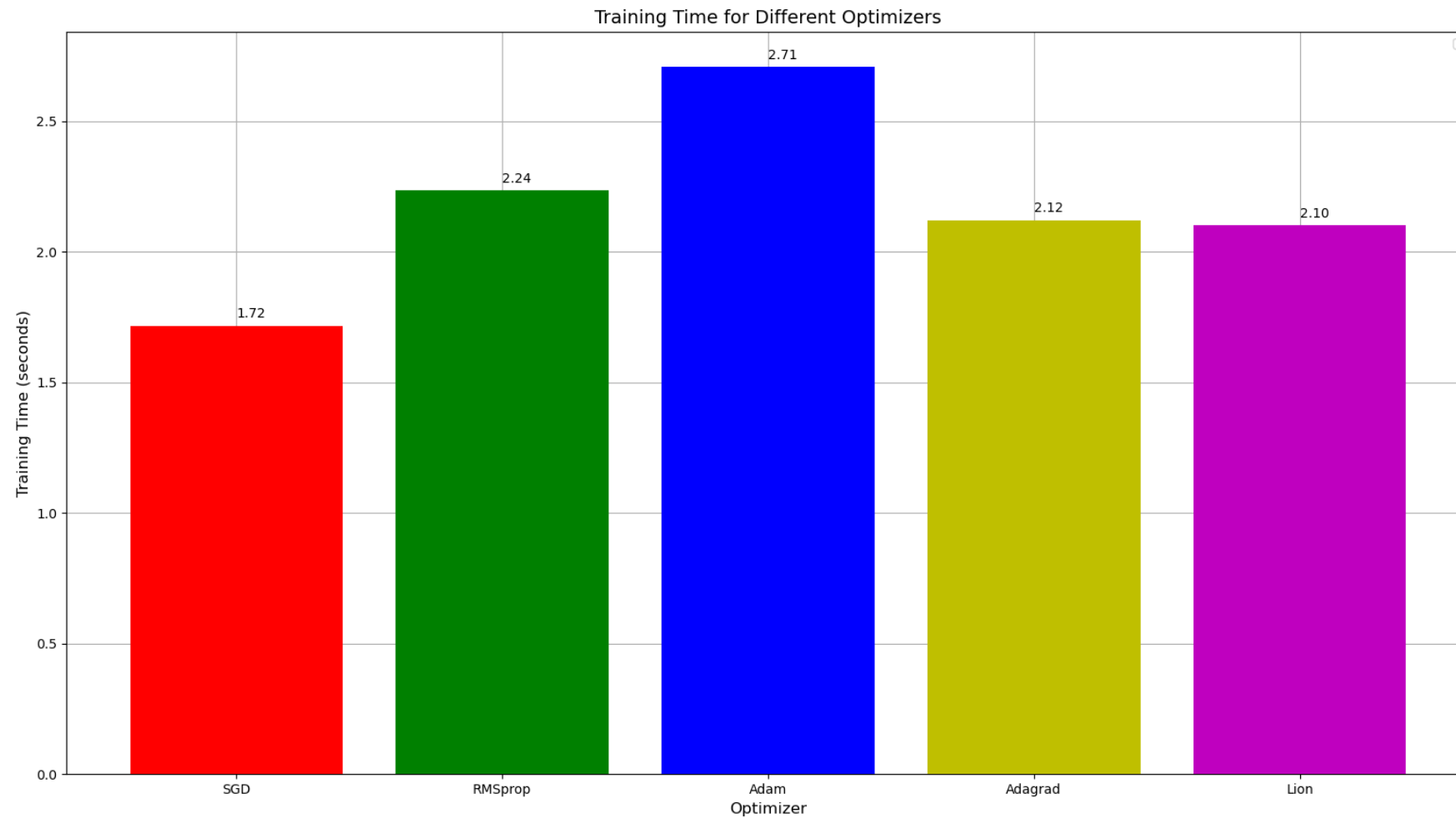
plt.show()
```

Test Set으로 모델을 평가하고, 최종 정확도와 오차, 소요 시간을 출력하는 부분이다.

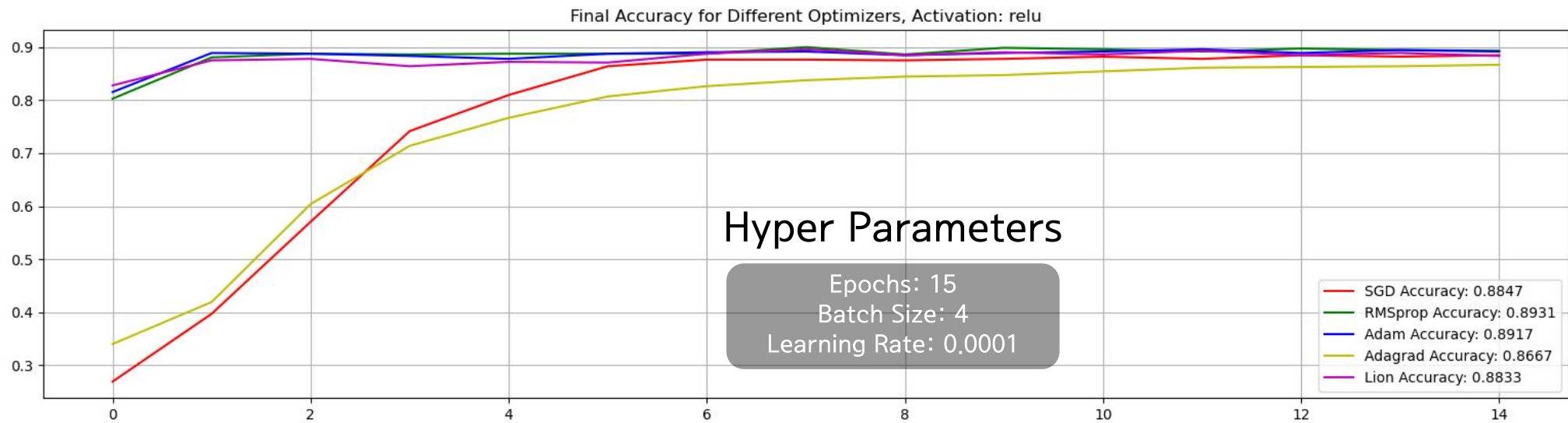
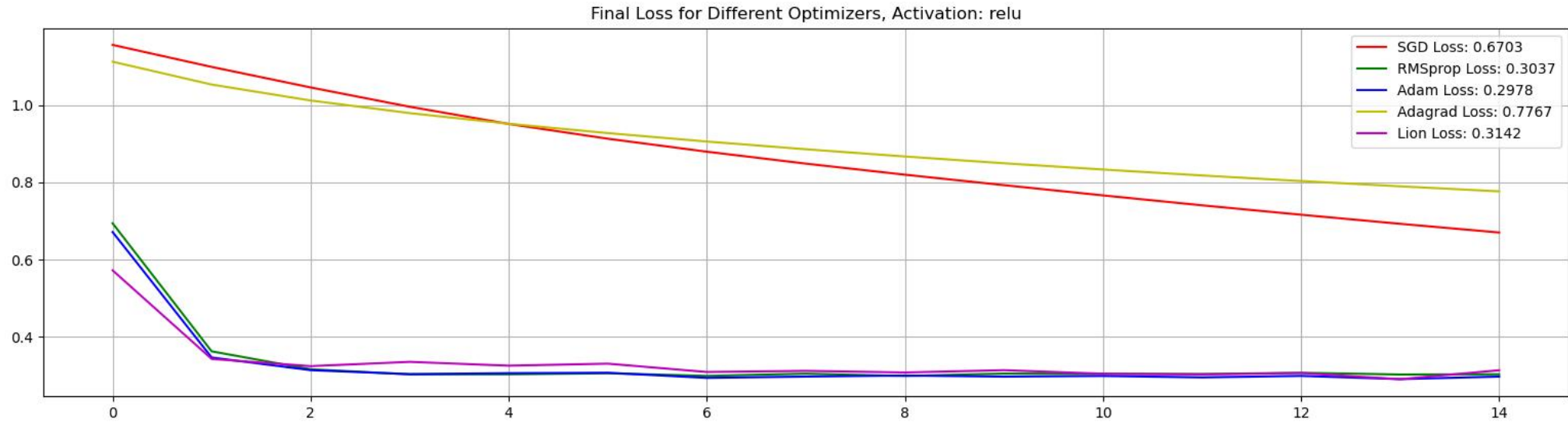
04 실습 #2 실행 결과



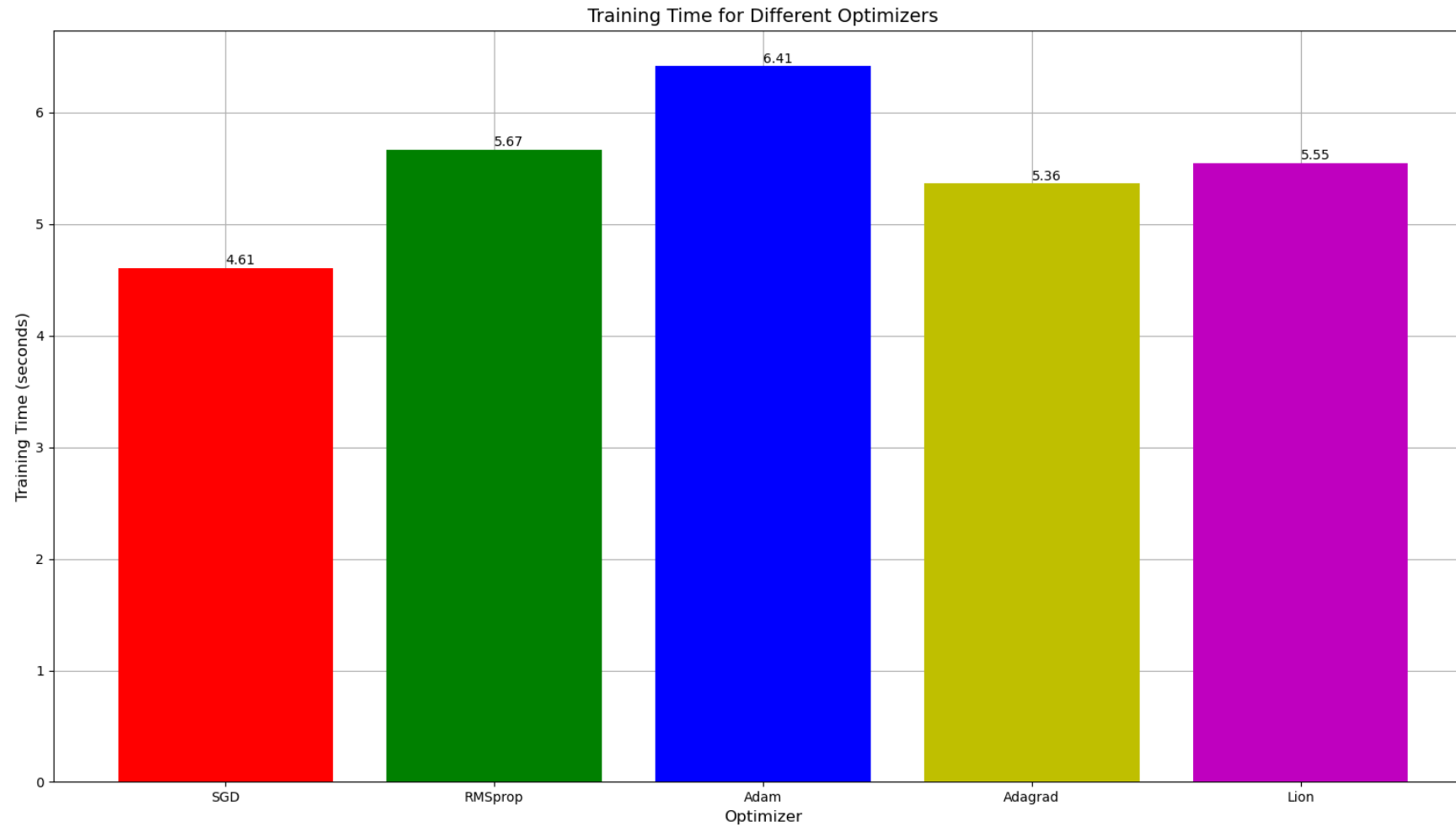
04 실습 #2 실행 결과



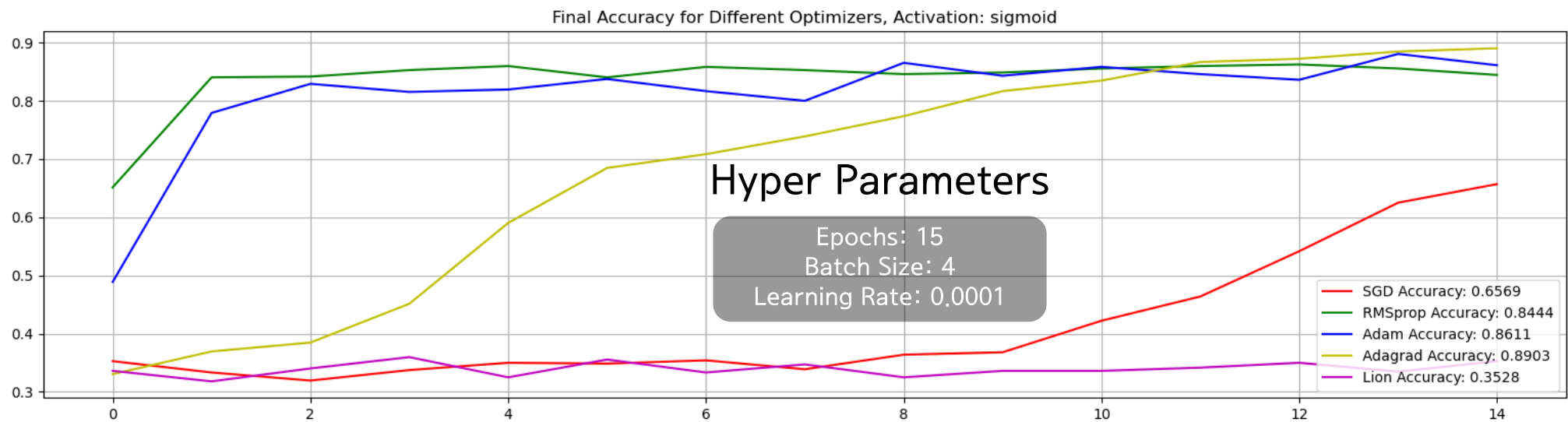
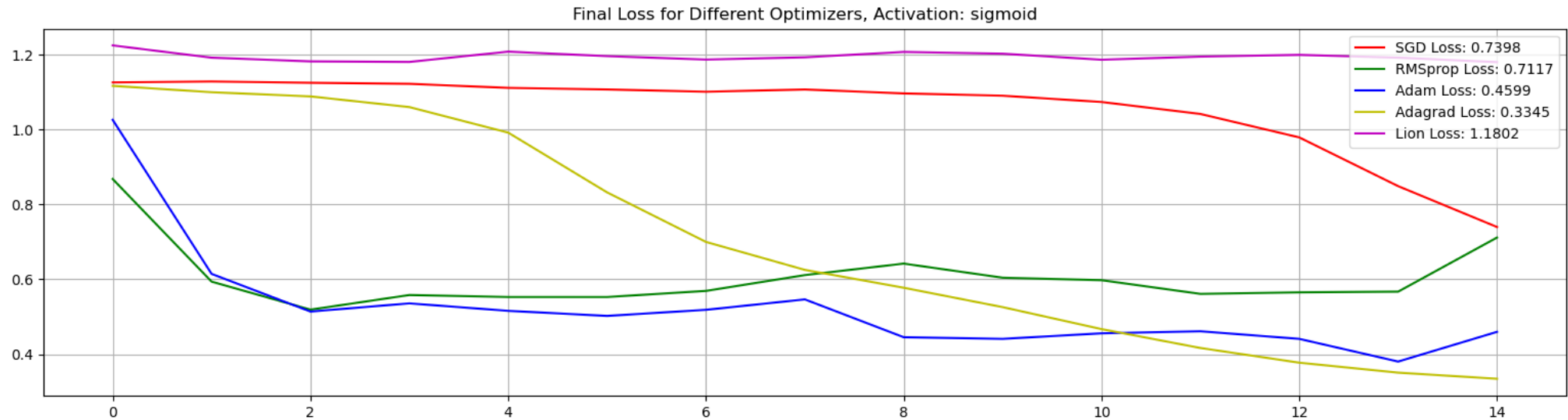
04 실습 #2 실행 결과



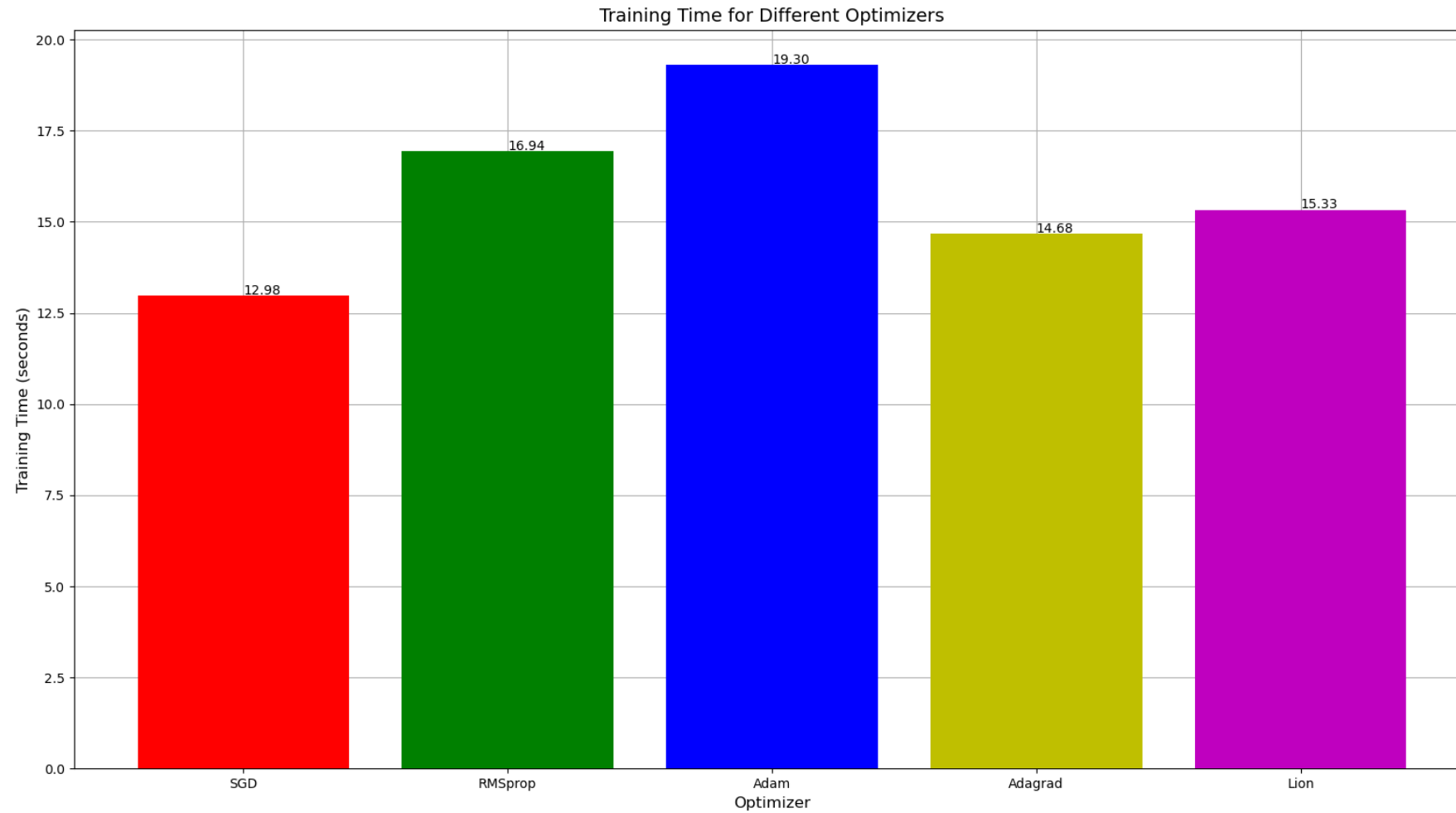
04 실습 #2 실행 결과



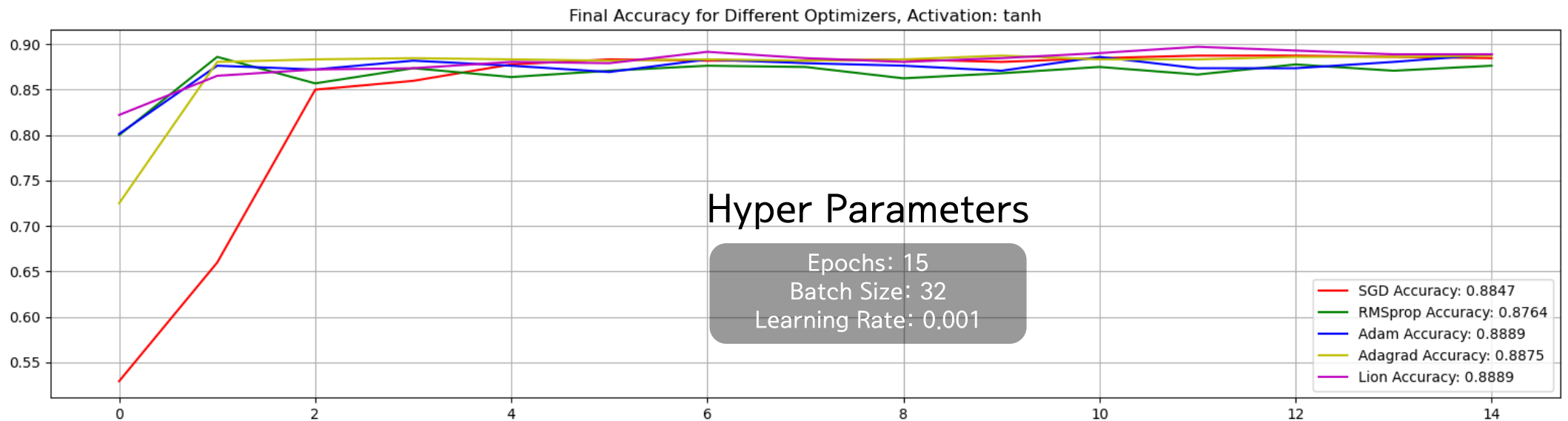
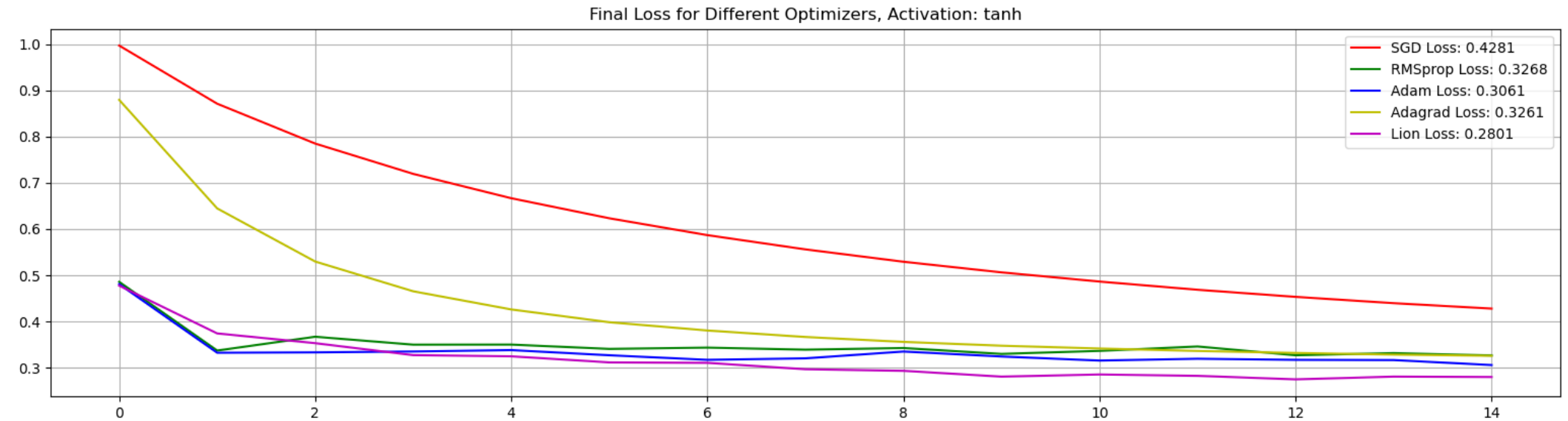
04 실습 #2 실행 결과



04 실습 #2 실행 결과



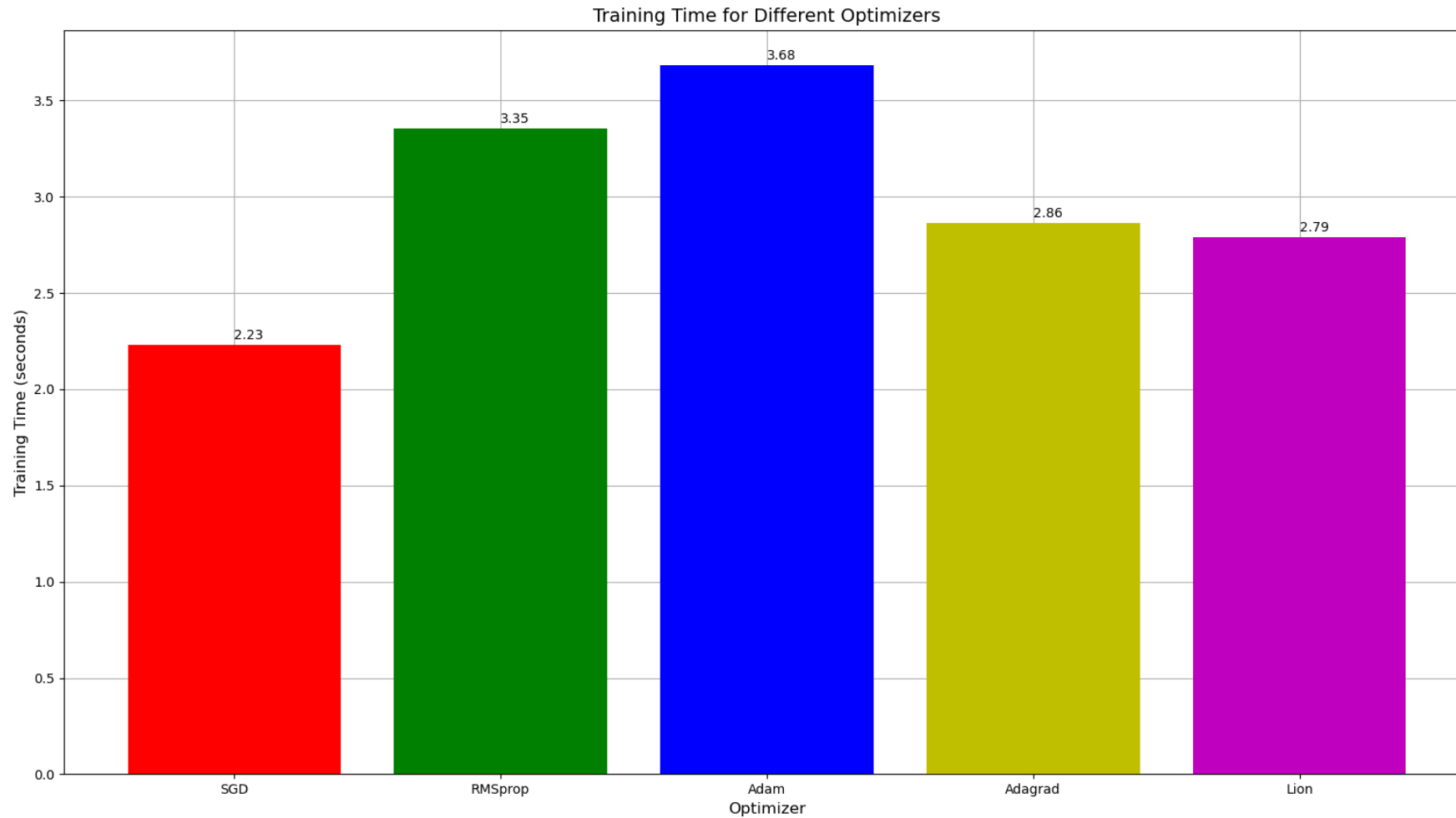
04 실습 #2 실행 결과



Hyper Parameters

Epochs: 15
Batch Size: 32
Learning Rate: 0.001

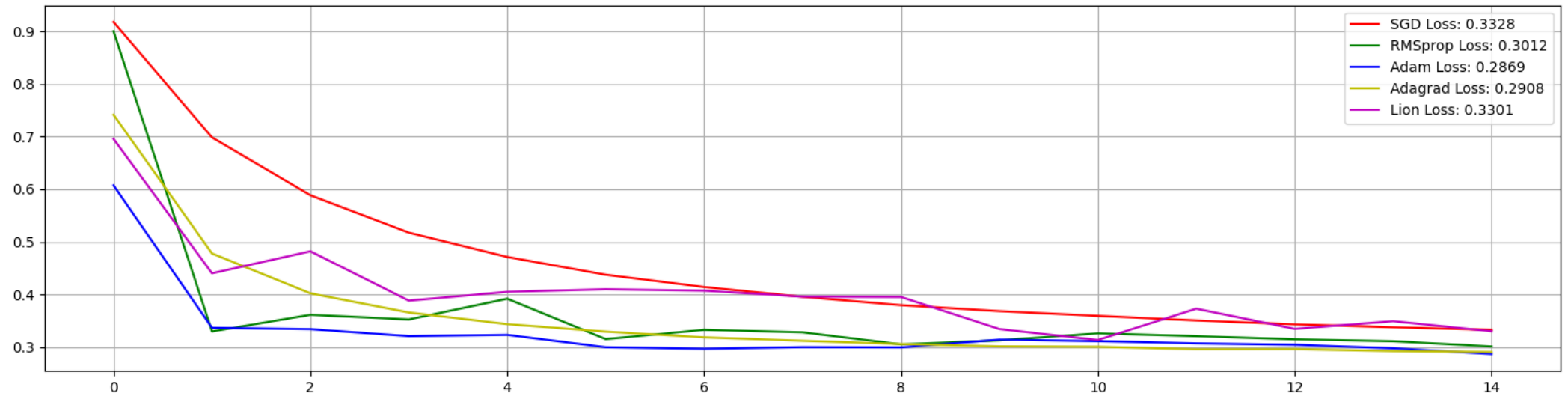
04 실습 #2 실행 결과



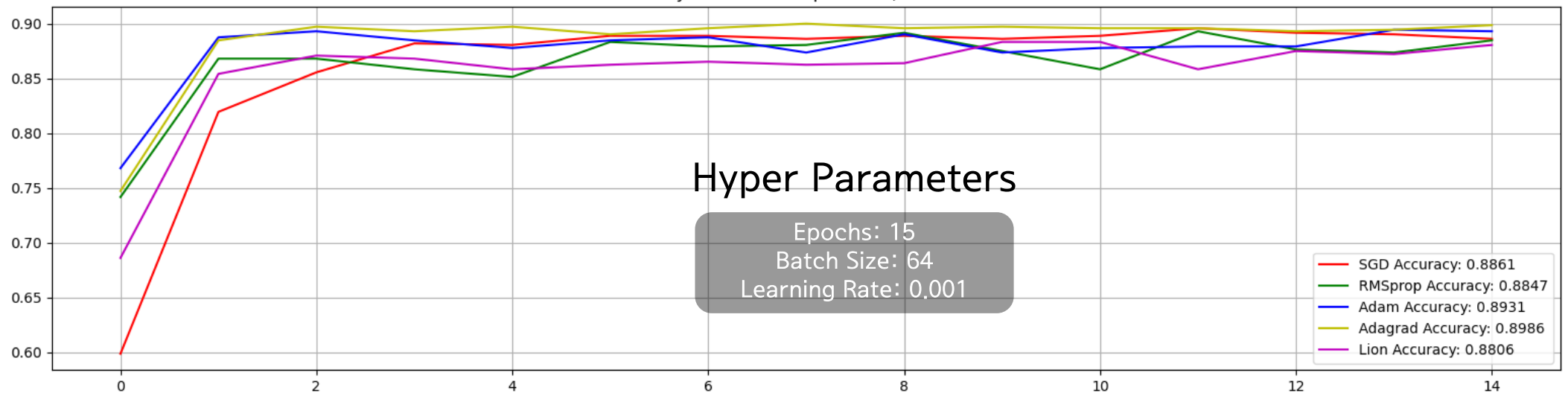
04 실습 #2 실행 결과

$$y = x$$

Final Loss for Different Optimizers, Activation: linear



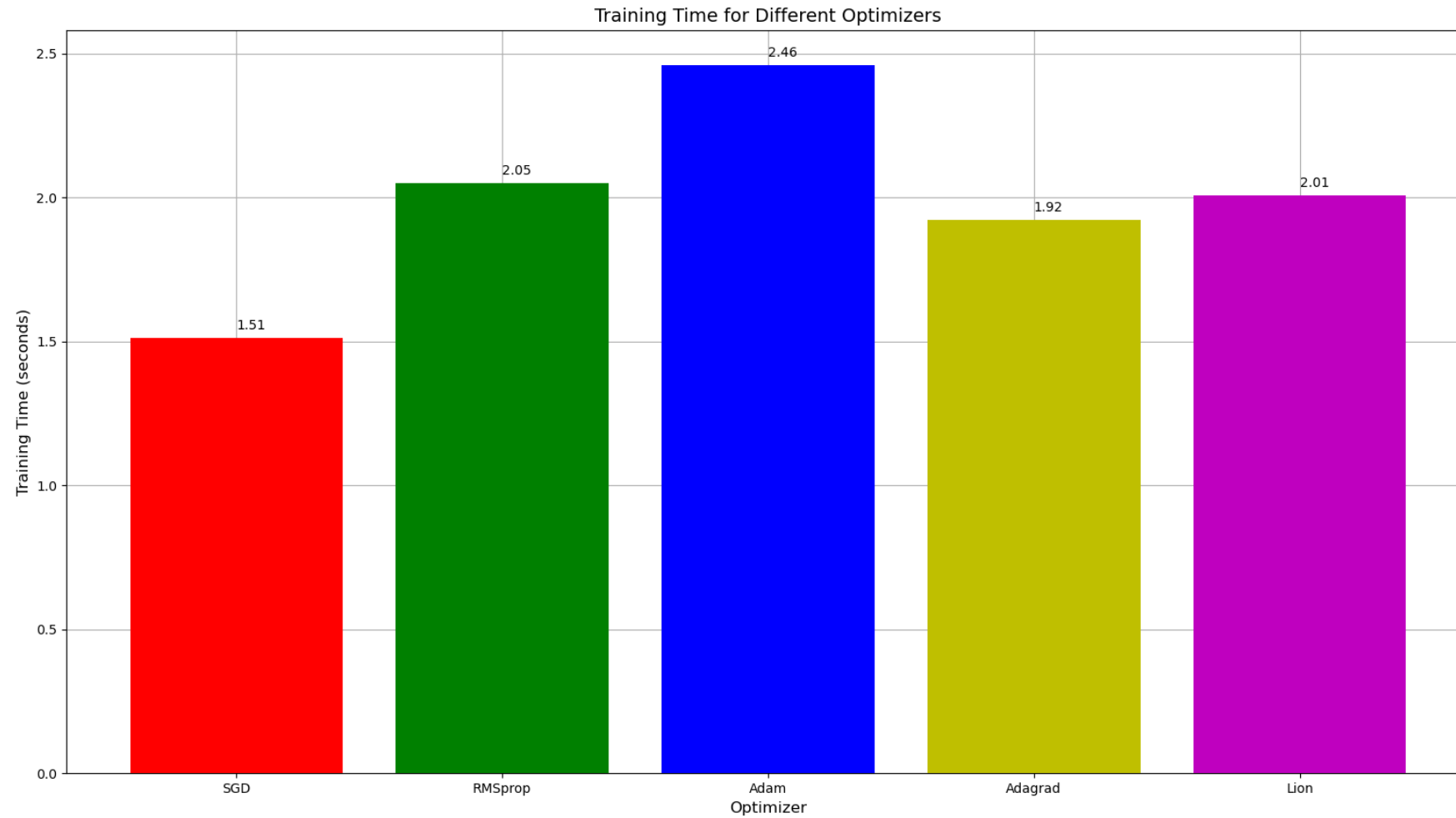
Final Accuracy for Different Optimizers, Activation: linear



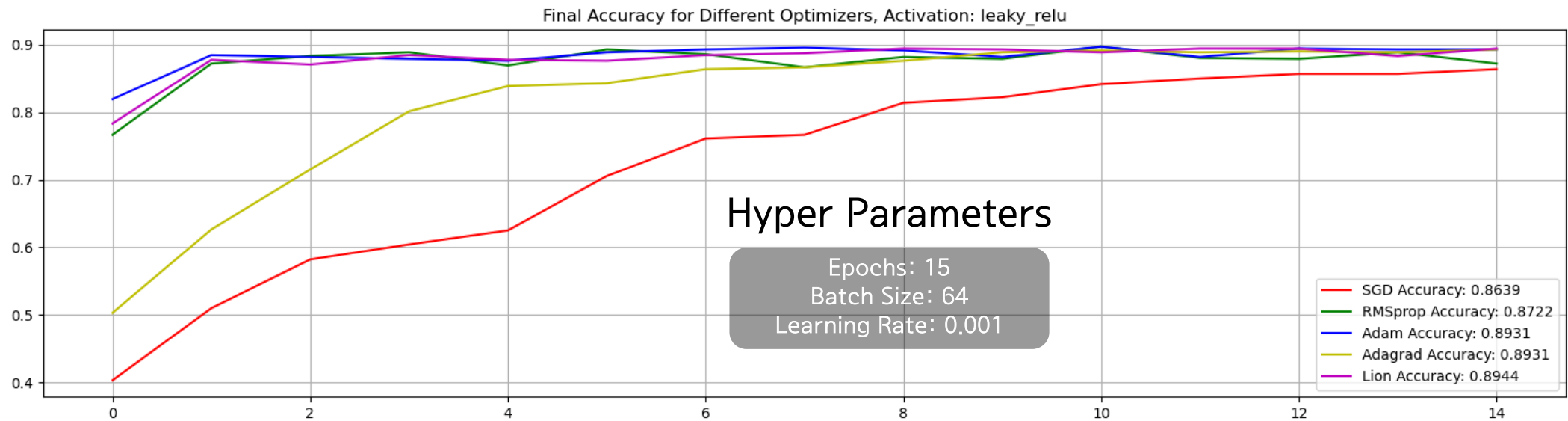
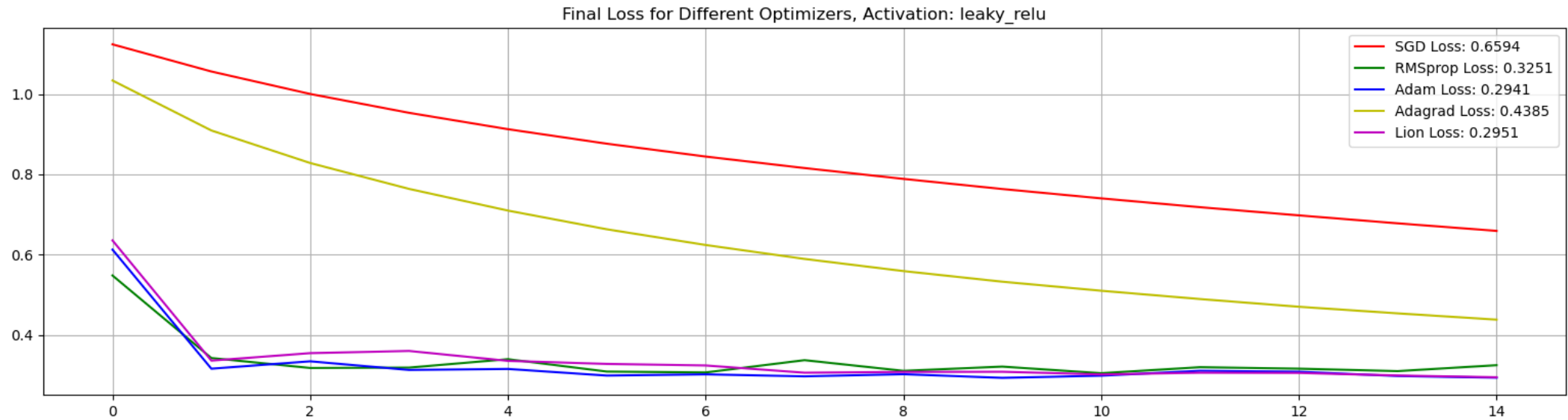
Hyper Parameters

Epochs: 15
Batch Size: 64
Learning Rate: 0.001

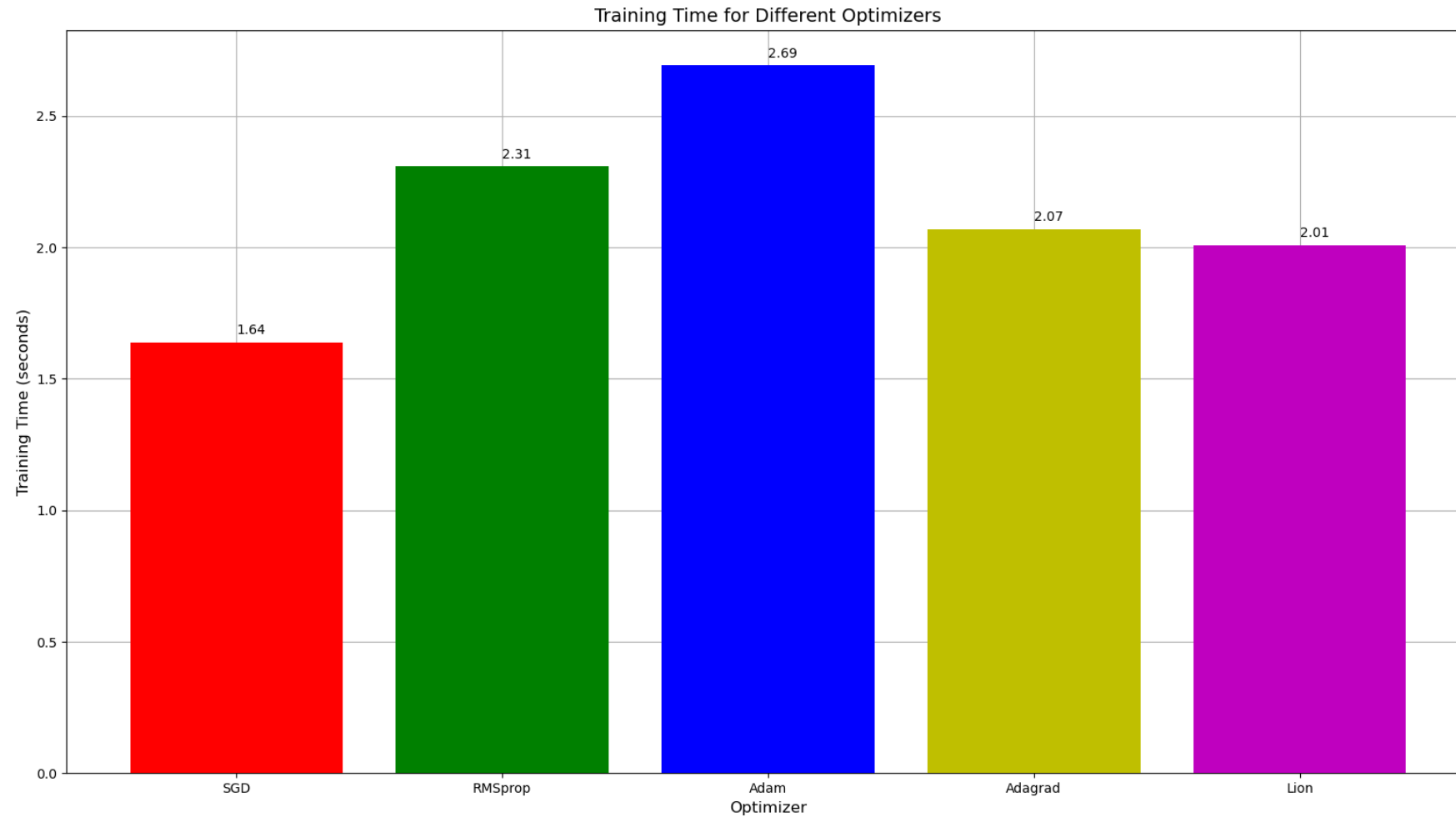
04 실습 #2 실행 결과



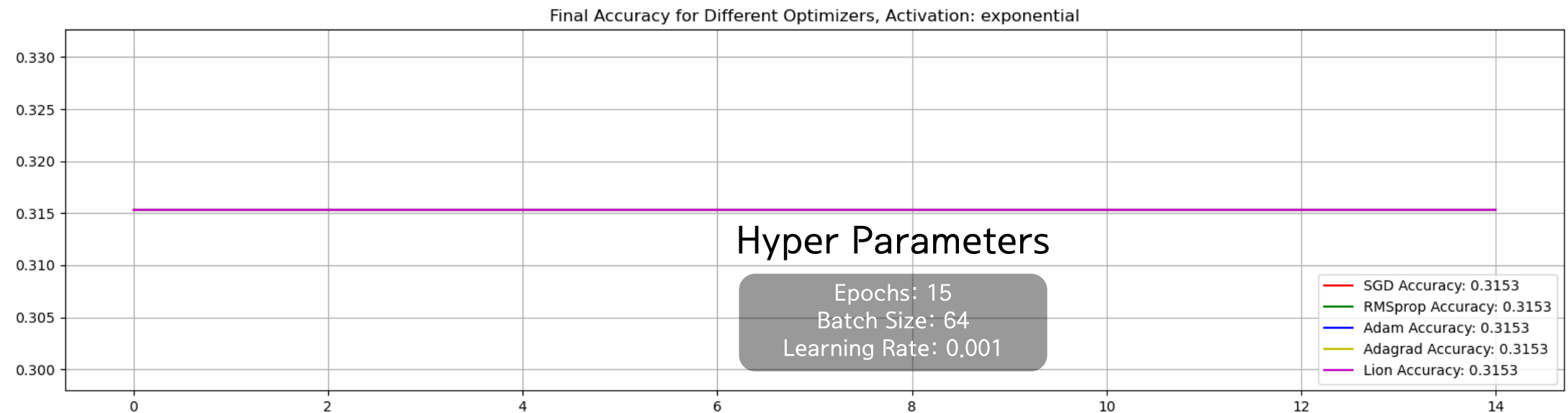
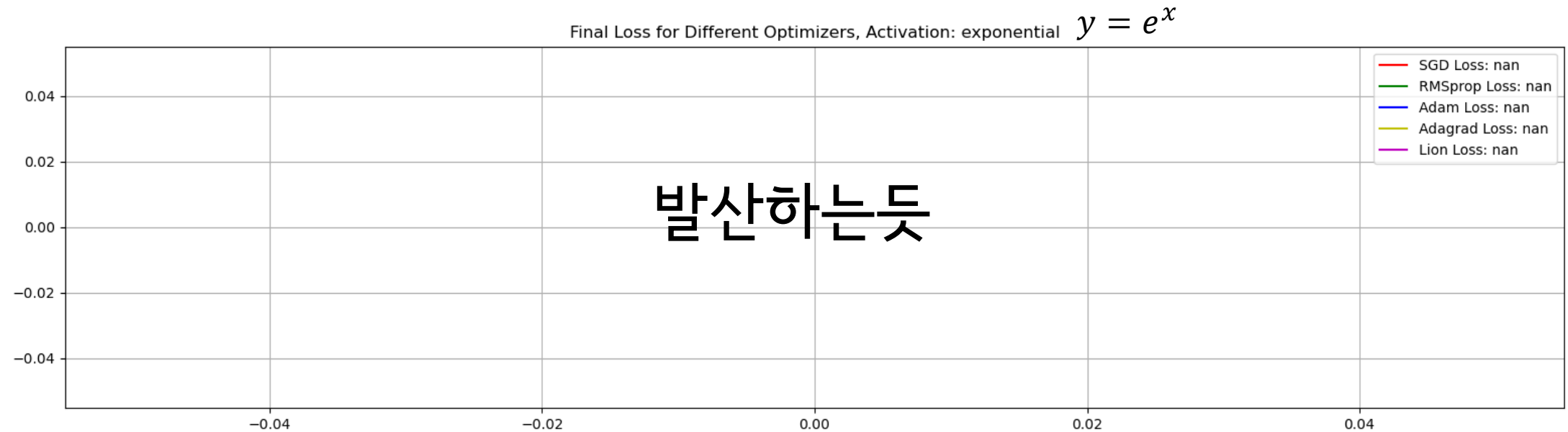
04 실습 #2 실행 결과



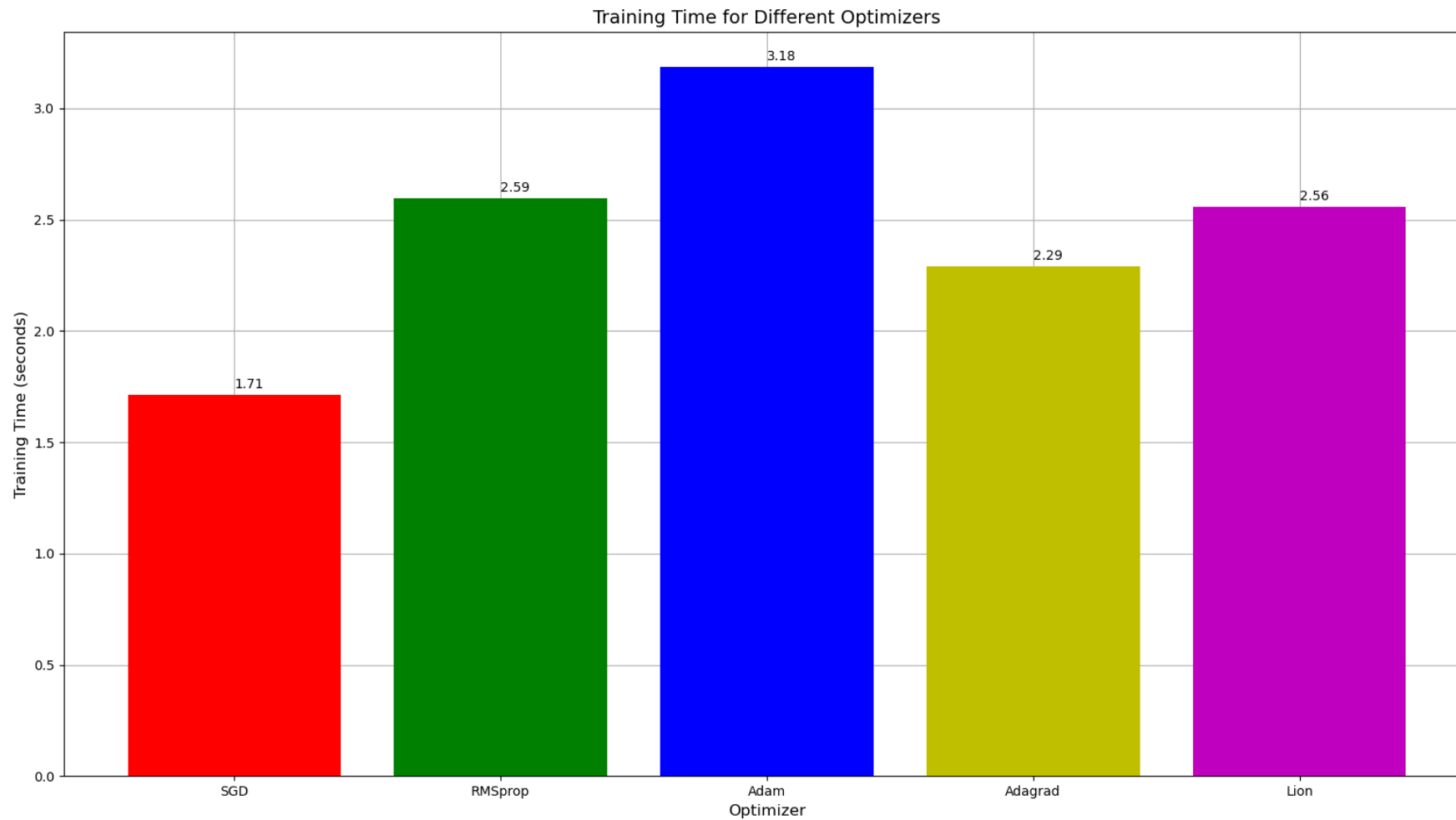
04 실습 #2 실행 결과



04 실습 #2 실행 결과



04 실습 #2 실행 결과



“NN_data”에 대한 최적의 옵티마이저

- 마지막의 NaN을 제외하고, 6번의 시행에서 가장 정확도가 높으면 5점, 가장 낮으면 1점을 줬을 때, 아래는 각 옵티마이저의 점수와 최종 순위이다.


Adam
(25)


Adagrad
(22)


Lion
(19)


RMSprop
(14)


SGD
(13)

- 이로 인해, “NN_data”에는 Adam이 가장 적합한 옵티마이저임을 알 수 있으며, 그 뒤로 Adagrad, Lion, RMSprop, SGD 순으로 최적임을 알 수 있다.
- 반면 소요 시간은 대체적으로 Adam < RMSprop < Lion $\approx$ Adagrad < SGD 순이었는데, 이는 Adam의 계산이 제일 복잡하고, SGD의 계산이 제일 단순하다는 것을 나타낸다.
- 만약 소요 시간까지 고려한다면, “NN_data”에서는 Adam보다는 Adagrad가 더 나은 선택이 될 것이다.

고급머신러닝 보고서
감사합니다